Course Overview and Learning Path

What you'll achieve in this course

This course teaches a practical, evidence-driven approach to web application security testing using AI agents as assistants—not replacements—for human judgment. By the end of the course you will be able to:

- Run safe, repeatable tests inside an isolated lab environment and capture canonical evidence.
- Turn observations into testable hypotheses and validate them with classic tools (Burp, curl, scanners) and reproducible workflows.
- Produce a concise, tiered deliverable: an executive summary, technical findings mapped to OWASP/CWE, and an evidence appendix with reproduction steps and confidence scoring.

You will work in a preconfigured lab (Kali + browsers, Burp as the canonical proxy) and submit artifacts such as a lab notebook, endpoint inventory, hypothesis plan, captured request/response pairs, and a mini pentest report.

The O → H → V → D workflow (Observe → Hypothesize → Validate → Document)

This four-step loop is the backbone of every engagement in the course. Treat it like a checklist you follow for each potential issue.

1. Observe

- Gather raw data from reconnaissance and passive monitoring: nmap, banners, Burp traffic captures, sitemaps, and agent-suggested leads.
- Capture canonical artifacts: full HTTP request/response pairs, headers, cookies, and timestamps. These are the single source of truth for later validation and reporting.

2. Hypothesize

- Convert an observed pattern into a concise, testable hypothesis (what you think is happening, why, and where it applies).

- Define the minimal conditions that would prove or disprove the hypothesis and list any assumptions, required credentials, or session state.

3. Validate

- Design non-destructive validation steps first. Prefer proof-of-concept checks that avoid data loss or business impact.
- Use Burp Repeater/Intruder, targeted curl requests, and controlled scanners as confirmatory tools. Record the exact requests and expected success/failure criteria.
- If automated agents propose tests, require a human confirmation gate before running anything potentially invasive.

4. Document

- For each validated finding, record: description, reproduction steps (exact requests), evidence screenshots or HTTP pairs, CWE/OWASP mapping, confidence level, and recommended remediation.
- Include retest steps and a short remediation validation plan so developers can verify fixes.

Following this loop keeps testing focused, reproducible, and defensible.

Evidence-first principle

Always attach concrete, canonical evidence to any claim. Statements without captured requests/responses or deterministic repro steps are treated as hypotheses, not findings.

Expected outcomes and how you'll be assessed

Assessments measure both process and product. The instructors will look for:

- Completeness of evidence: Are the canonical HTTP pairs, headers, and timestamps present and clearly labeled?
- Reproducibility: Can an assessor follow the reproduction steps and observe the same behavior using the provided artifacts?

- Correctness of mappings: Is each technical behavior correctly mapped to an OWASP Top 10 entry and an appropriate CWE with a short rationale?
- Confidence and assumptions: Does the submission clearly state confidence levels and any assumptions or test limitations?
- Safety and scope adherence: Were tests confined to authorized targets and non-destructive unless explicitly permitted?
- Communication quality: Are the executive summary and remediation suggestions concise, prioritized, and actionable for stakeholders?

Rubric (high level):

- Evidence & Repro (40%): Canonical artifacts and clear reproducibility steps.
- Technical Analysis (25%): Correct hypotheses, validation approach, CWE/OWASP mapping.
- Remediation & Impact (15%): Practical fixes, prioritized by business impact.
- Process & Safety (10%): Scope adherence, non-destructive testing, human review gates.
- Presentation (10%): Executive summary clarity and appendices organized for reviewers.

Deliverables you will submit include the lab notebook with endpoint inventory, a hypothesis plan, captured evidence bundles, and a mini pentest report. Each deliverable is expected to follow the course templates for naming, timestamps, and evidence linking.

If you're unsure about whether a test is allowed or if an agent's suggestion looks risky, pause and ask. Safety, authorization, and reproducibility are grading-critical and non-negotiable.

Provisioning the Lab (Kali + Browsers)

Getting your Kali workstation and browsers ready is the first practical step. The goal is a reproducible, isolated environment you can snapshot and restore quickly, with browser profiles and tooling configured for safe testing.

Why this matters

A poorly provisioned lab leaks state, secrets, or network traffic and makes results hard to reproduce. A good lab is: isolated from production and your host, snapshottable, and contains the minimal tools and browser profiles you need for interactive testing.

Quick provisioning steps

1. Obtain a clean Kali image

- Download the official Kali VirtualBox/VMware image or install Kali on a VM. Use vendor-provided images to avoid extra configuration drift.
- Create one VM instance per tester or per experiment to avoid cross-contamination.

2. Virtual machine networking: isolate, but allow controlled updates

- Minimum recommended network adapters:
  - Adapter A: NAT (or NAT Network) — allows package updates and occasional internet connectivity.
  - Adapter B: Host-only or Internal Network (name it something like `lab-net`) — the lab-facing adapter you use to connect to target VMs or test networks.
- If you need stricter isolation, use Host-only/Internal only and provide a separate jump VM that has restricted NAT for updates.
- Avoid bridged networking unless you understand the security implications: bridged exposes the VM directly to your LAN.

3. Snapshotting and state management

- Immediately take a base snapshot after finishing OS updates and package installs. Name it clearly: `base-YYYYMMDD` and store a short note of what it contains.
- Before running any active or destructive tests, create a new snapshot (e.g., `pre-scan-YYYYMMDD`). Revert to that snapshot to restore a known-good state.
- Keep snapshots small and purposeful; delete or export old appliances as part of routine housekeeping.

Snapshot discipline

Take a snapshot before you change configurations or run any active tests. Reverting to a snapshot is the fastest way to ensure reproducibility and to recover if something breaks.

4. Install and configure browsers and core utilities

- Install lightweight browser builds for testing and separate profiles for different roles (recon, authenticated, privileged):
  - Example install commands (run as root or via sudo):

```
apt update && apt upgrade -y
apt install -y firefox-esr chromium curl git net-tools
```

- Create distinct browser profiles and disable synchronization and password storage in each profile. Name them clearly (e.g., `kali-recon`, `kali-auth`).
- For each profile: disable auto-fill, disable password manager, and clear cache/cookies between sessions where appropriate.

5. User accounts and privileges

- Use a non-root local user for day-to-day testing where possible. Keep the root account available for administrative tasks only.
- Create test accounts that mirror application roles (user, admin) when the lab target supports them; use those accounts for role-based testing rather than using a full-privilege account.

6. Tool installation basics

- Install the tools you'll use in the course (Burp is the canonical proxy used later; you can install the Community or Professional edition separately):
  - `curl`, `nmap`, `sqlmap`, `nikto`, and the browsers are a good starting set.
- Keep tools up to date but test updates against the base snapshot so you can roll back if an update breaks a workflow.

7. Test connectivity to the lab target

- Verify basic network reachability and HTTP behavior against the canonical lab target:

```
# check HTTP headers
curl -I http://testphp.vulnweb.com/
```

```
# fetch the home page and show the first few lines
curl -s http://testphp.vulnweb.com/ | head -n 20
```

Expected checks:

- You should receive an HTTP 200 response (or a consistent redirect if configured).
- The response body should contain recognizable application text (useful for confirming you hit the intended test site).
- If you route traffic through a proxy (Burp) later, confirm that proxy interception and TLS interception (for HTTPS) work as expected.

Best practices and hardening

- Browser profiles: maintain one profile dedicated to instrumentation (with proxy set to 127.0.0.1:8080) and another for exploratory browsing without instrumentation.
- Secrets: never enable browser sync or save passwords in test profiles. Remove any API keys or host credentials from snapshots.
- Logging and artifact names: adopt a consistent naming scheme for snapshots, evidence bundles, and exported artifacts (include date, tester initials, and purpose).
- Backups: export a clean appliance file (`.ova`) after creating the base snapshot so you can distribute a known-good starting image.

Checklist — verify before you start testing

- VM created from an official Kali image and updated
- Two network adapters configured (NAT + Host-only/Internal)
- Base snapshot taken and exported
- Browsers installed with isolated profiles; sync and password saving disabled
- Non-root user created for testing tasks
- Core tools installed (`curl` , `nmap` , etc.)
- Confirmed HTTP reachability to http://testphp.vulnweb.com/ and recorded the curl output

If all items are green, you're ready to configure Burp and begin the Burp-first evidence capture workflow covered in the following lesson.

Burp Suite as the Single Source of Truth

Why Burp as the single source of truth

Burp Suite should be your canonical evidence store during interactive testing. It gives you a single, timestamped record of every proxied interaction (requests, responses, headers, cookies, TLS details, and site map entries) that you can inspect, replay, annotate, and export. The rest of your workflow—hypothesis formulation, validation steps, and report evidence—should reference Burp artifacts so results are reproducible and auditable.

Configure your browser to use Burp (safe, repeatable profiles)

1. Use dedicated browser profiles for testing. Disable sync, password saving, and autofill in those profiles.
2. Configure the proxy to point at Burp:
   - Proxy: `127.0.0.1`
   - Port: `8080`
3. Install Burp's CA certificate into the test profile so HTTPS can be intercepted cleanly:
   - Firefox: Import the certificate into the profile's Certificate Authorities (Preferences → Privacy & Security → View Certificates → Authorities → Import).
   - Chromium-based browsers: Import into the OS certificate store or use the browser's certificate settings for the test profile. Alternatively, use a dedicated profile that trusts the Burp CA.

Tip: Keep one profile configured to route traffic through Burp and another profile for non-proxied browsing to avoid accidental leakage of sensitive session state into your test traces.

Intercepting and capturing canonical evidence

What to capture for every meaningful interaction:

- Full raw HTTP request and response bodies (not just parsed views).
- All request and response headers, including `Cookie`, `Set-Cookie`, `Authorization`, `Content-Type`, and `Content-Length`.
- Timestamps and the observed server IP/port (visible in HTTP history and site map).
- Any TLS details (certificate chain, negotiated version/cipher) when relevant.
- Context: the user account or role used, test steps taken immediately before the captured exchange.

Practical steps in Burp

- Proxy → Intercept: Use interception to observe and, when necessary, modify requests before they leave the browser. Only intercept when you need to; otherwise, leave interception off and rely on HTTP history.
- Proxy → HTTP history: This is your timeline. Right-click entries to "Show request in browser", send to Repeater, or save the item.
- Target → Site map: Use this to create an endpoint inventory and to mark what is in-scope.
- Repeater & Intruder: Send canonical requests to Repeater to craft reproducible proof-of-concept checks. Always save the exact request you used.

Naming and metadata conventions (short, consistent)

- Include date (YYYYMMDD), tester initials, endpoint path, and a short purpose: `20251201_JD_GET_/login_capture.raw`.
- Add annotations in Burp's comments field to link a request to a hypothesis or finding ID.

Exporting and sharing Burp artifacts

Options to persist or share evidence:

- Project files: Save the entire Burp project (`.burp`) when you need to preserve everything for later analysis. This preserves site map, history, and settings.
- Export selected history items: In Proxy → HTTP history, select items → right-click → "Save selected items". Use Burp's XML format if you need a machine-readable canonical export.
- Export requests/responses to files: Use the "Copy to clipboard" → "Copy request/response" or save the raw messages to files. Keep raw pairs together and label them clearly.
- When feeding artifacts into agents or external tools, export minimal, well-labeled bundles containing only the items needed for the check.

Session handling, cookies, and auth tokens

Principles

- Treat authentication material as sensitive evidence. Avoid storing real credentials in shared artifacts; prefer steps to reproduce authentication instead.
- Capture the authentication flow (login request, responses, subsequent session cookie/authorization header) as canonical evidence, but redact secrets before distributing artifacts beyond evaluators who need them.

Burp features to manage sessions

- Cookie jar: Burp automatically stores cookies. Use the Target → Sessions and Proxy → Options cookie features to inspect what was stored.
- Session handling rules: Create rules to handle login flows automatically (useful for automated scans or repeatable tests). A rule can perform login requests and extract tokens into later requests.
- Macros: Build macros to re-establish sessions or perform multi-step authentication during repeatable runs.

Practical tips

- Reproduce auth tokens rather than sharing them. For developer handoff, describe the exact login request (with test account), then provide the captured cookie header with placeholders (e.g., `SESSIONID=<redacted>`), and include steps to obtain a fresh token.
- If a test requires sharing tokens with trusted reviewers, keep a separate redaction/non-redaction artifact set and a clear retention and deletion plan.

Scope rules and safety controls

Always define a strict scope in Target → Scope before you begin active testing. Steps:

1. Add target hosts and paths explicitly to the scope.
2. Enable "Intercept only in-scope" (Proxy → Options → Intercept Client Requests and Server Responses rules) and check "Only proxy traffic in scope" where relevant.
3. Configure Scanner and Intruder to respect scope so automated checks won’t touch out-of-scope hosts.

Use scope to enforce non-destructive behavior:

- Keep sensitive administrative consoles, third-party hosts, and analytics endpoints out of scope.

- Use the "Site map" view to audit what Burp has observed and remove any accidental out-of-scope entries.

Redaction, retention, and evidence hygiene

- Redact secrets before sharing. Replace tokens, passwords, and PII with clear placeholders and keep an internal mapping table that is not shared.
- Retain raw `.burp` or raw message files only as long as required by the engagement and delete/export per your evidence retention policy.
- When attaching evidence to findings, include both the redacted artifact and reproducible steps that allow a reviewer to re-create the same request/response pair in their own lab.

Checklist — quick pre-test sanity checks

- Dedicated browser profile configured for Burp with CA installed
- Target scope defined and verified in Target → Scope
- Snapshot or baseline state captured (VM snapshot) before active testing
- Evidence naming convention chosen and documented for this engagement
- Session handling rules or macros created if you need repeatable authentication flows

Safety: stay in-scope

Never run active or destructive tests against hosts or endpoints that are not explicitly in scope. Use Burp's scope controls and browser profile separation to prevent accidental leakage or impact.

Closing note

Treat Burp as the evidence backbone: capture raw requests/responses, timestamp and tag them thoughtfully, manage session artifacts carefully, and always enforce scope. Doing so makes your testing repeatable, defensible, and easy to communicate to developers and assessors.

Installing AI Tools

Installing and Securing AI Tools

This lesson shows how to install common AI clients safely and how to configure them so they fit into a defensible pentest workflow. The emphasis is on minimizing blast radius: isolate tools, protect secrets, control data flows, and prefer ephemeral / least-privilege access.

1) Install with containment in mind

- Prefer installing AI clients inside an isolated environment (dedicated VM, container, or virtualenv) that you snapshot before experimenting. Avoid installing directly on a shared workstation.
- Use language-appropriate virtual environments: `python -m venv .venv` or containers (`docker`) to keep dependencies isolated and reproducible.
- Example (Python client installs):

```
# create and activate a virtualenv
python -m venv .venv && source .venv/bin/activate
# install common SDKs
pip install openai anthropic
```

- For vendor-specific CLIs (Gemini CLI, Cursor CLI, Claude/Anthropic tools), follow the vendor documentation but run installers inside the isolated environment and audit any post-install network or file-system behavior before entering sensitive work.

2) Secrets and API key handling (practical rules)

- Never hard-code API keys in scripts or commit them to source control. Treat keys like passwords.
- Store keys in a secrets manager when possible (HashiCorp Vault, cloud KMS/Secret Manager, or a local password manager). If you must use environment variables for local work, keep them in a file with strict permissions:

```
# .env file: permission 600
OPENAI_API_KEY="sk-..."
```

```
ANTHROPIC_API_KEY="claude-..."
chmod 600 .env
```

- Use short-lived, scoped keys for each project or engagement. Create per-project or per-session keys and rotate or revoke them after the engagement.
- Avoid exposing keys to GUI clients that automatically persist them to cloud-synced profiles. Prefer CLI clients that read keys from controlled locations.
- If you must share an API-powered artifact with a reviewer, provide redacted artifacts and reproduction steps rather than sharing live keys.

Proxy and logging

Route outgoing API requests through a local API proxy that can log, redact, and optionally block requests that contain sensitive patterns. A proxy gives you an audit trail and the ability to stop accidental exfiltration.

3) Local vs cloud memory and data flows

- Understand where model input and derived data go. Many hosted LLM services retain prompts, files, or conversation history by default unless you opt out.
- For sensitive evidence (requests/responses, credentials, PII), do not send raw artifacts to cloud models. Instead:
  - Redact secrets and PII before sending any evidence to a cloud model.
  - If you need full-context analysis, run an on-premise or local model when possible.
- Disable or clear long-term memory and conversation history features where available. Explicitly configure retention policies and deletion workflows for any stored interactions.

Data leakage risk

Do not paste raw HTTP requests, full cookie headers, or secret tokens into cloud chat windows or shared workspaces. Even ephemeral debugging snippets can contain sensitive data that moves outside your control.

4) Workspace separation and test accounts

- Create dedicated workspaces for each engagement. Separate production-facing accounts from lab/test workspaces at both the application and AI-tool levels.
- Use test accounts with the minimum required privileges for any authentication-related testing. Never use real user accounts or production admin credentials in lab experiments.
- Maintain a standard naming convention for workspaces, keys, and artifacts that encodes the engagement and date (e.g., `engagementX_openai_key_2025-12-01`). This makes rotation and cleanup easier.
- For browser-based AI clients or UIs, disable sync and automatic backups in the profile used for testing.

5) Tool-specific safe defaults (practical recommendations)

- Cursor AI / Editor-integrated tools
  - Limit file-system access to the project directory; avoid granting access to the full home directory or system secrets.
  - Turn off automatic uploading of files or telemetry; review upload prompts before approval.
- Gemini CLI (or other vendor CLIs)
  - Prefer authenticated sessions that use scoped keys and set short session lifetimes.
  - If the CLI offers a local mode (offline model or on-device execution), prefer it for sensitive data analysis.
- ChatGPT / OpenAI clients
  - Use per-project API keys, enable usage caps, and disable “save to history” or opt out of data retention when the option is available.
  - When sending evidence, redact tokens and PII and send only structural summaries or synthetic reproductions.
- Claude / Anthropic clients
  - Use model-specific privacy options and request explicit non-retention if supported.
  - Use the HTTP client in isolated environments rather than public UIs for scripted, auditable runs.

6) Operational hygiene: logging, retention, and cleanup

- Keep an access and usage log for all keys and workspace sessions. Record who requested a key, why, and when it was revoked.
- Automate key rotation and enforce expiration for all non-human (service) keys.

- After an engagement, remove keys, delete ephemeral workspaces, and purge any stored conversation history or uploaded files according to your retention policy.

7) Human-in-the-loop controls and approvals

- Any potentially invasive or destructive test proposed by an AI should require an explicit human approval step before execution. Implement guardrails in your scripts or CI to block automatic execution without a signed approval.
- Keep a checklist for approval: scope confirmation, snapshot taken, estimated impact, rollback plan, and required monitoring.

8) Quick verification checklist (run these before first use)

- Is the AI client installed inside an isolated environment (VM, container, or venv)?
- Are API keys stored in a secrets manager or a file with strict permissions?
- Do keys have per-project scope and short lifetimes?
- Is API traffic routed through a proxy or gateway that logs and redacts sensitive fields?
- Are workspaces separated per engagement and sync disabled for test profiles?
- Have you disabled or configured model memory/retention options appropriately?
- Do you have a documented approval step for any automated active tests?
- Is a snapshot (VM or container image) taken so you can revert after tests?

Follow these steps to keep AI tooling productive without creating avoidable risks. When in doubt, err on the side of containment: redact, isolate, and require explicit approvals before any action that could impact scope, privacy, or production systems.

Gemini-CLI on Kali Linux

Overview

Gemini-CLI is an open-source AI command-line interface that lets you use Google’s Gemini models directly from your terminal. It can be installed on Kali Linux via the apt package `gemini-cli`.

---

1. Update System

Update package lists and upgrade existing packages:

```
sudo apt update
sudo apt upgrade -y
```

Install recommended prerequisites (Node.js is a dependency for Gemini-CLI through apt):

```
sudo apt install -y nodejs
```

---

2. Install Gemini-CLI

On Kali Linux (2025.3 and later), the `gemini-cli` package is available directly from the Kali repositories:

```
sudo apt install gemini-cli
```

Verify installation:

```
gemini --help
```

If the CLI starts, installation succeeded. (Kali Linux)

---

3. Gemini Authentication Methods

Gemini-CLI requires authentication before use. There are multiple methods:

- **Google login (OAuth)**
- **Gemini API key**
- **Vertex AI / Google Cloud authentication**

This guide uses the **Gemini API key** method. (Gemini CLI)

Gemini-cli Interface

3.1 Generate an API Key

1. Open Google AI Studio: https://aistudio.google.com/app/apikey
2. Create a new API key.
3. Copy the key.

---

4. Configure API Key

4.1 Set Environment Variable

In your terminal (bash):

```
export GEMINI_API_KEY="YOUR_API_KEY"
```

Replace `YOUR_API_KEY` with your actual key.

4.2 Persistent Configuration

To make it permanent (applies on every shell start):

```
echo 'export GEMINI_API_KEY="YOUR_API_KEY"' >> ~/.bashrc
source ~/.bashrc
```

Confirm the variable is set:

```
echo $GEMINI_API_KEY
```

If it prints your key, it is configured correctly.

Alternative configuration methods: placing the key in a `.env` file at `~/.gemini/.env` (the CLI loads it automatically). (Philschmid)

5. Running Gemini-CLI

5.1 Basic Non-Interactive Prompt

You can run a single prompt directly:

```
gemini -p "Explain the difference between SQL injection and XSS"
```

or without `-p` (positional prompt):

```
gemini "Explain the difference between SQL injection and XSS"
```

5.2 Interactive Mode

Start an interactive session:

```
gemini
```

Inside the CLI you can enter prompts and receive responses dynamically.

5.3 Model Selection

Specify a model (if supported):

```
gemini -m gemini-2.5-pro -p "Generate a secure Python hash function"
```

Replace `gemini-2.5-pro` with another model if needed.

6. Advanced Authentication (optional)

6.1 Vertex AI Authentication

If you use **Vertex AI** services instead of the API key, set additional environment variables:

```
export GOOGLE_CLOUD_PROJECT="your_project_id"
export GOOGLE_CLOUD_LOCATION="your_region"
```

Then authenticate with `gcloud`:

```
gcloud auth application-default login
```

Start Gemini-CLI and choose Vertex AI authentication mode. (Gemini CLI)

---

7. Configuration File

Gemini-CLI supports config files. Create directory:

```
mkdir -p ~/.config/gemini
```

Create a YAML config (optional):

```
# ~/.config/gemini/config.yaml
api_key: "YOUR_API_KEY"
model: "gemini-2.5-pro"
temperature: 0.2
max_output_tokens: 2048
```

Save and exit. The CLI may pick up settings from this file.

---

8. Updating or Removing

Update

```
sudo apt update
sudo apt upgrade gemini-cli
```

Uninstall

```
sudo apt remove gemini-cli
```

Prompting Foundations for Pentest Agents

This lesson explains how to craft **effective prompts** for AI-assisted penetration testing agents. The focus is on templates for different phases of a pentest workflow (reconnaissance, hypothesis formulation, and reporting), as well as how to avoid **open-ended destructive instructions** that could lead to unsafe actions.

Section 1 — Why Prompt Engineering Matters in Security

Prompt engineering is the discipline of creating precise instructions that guide a large language model (LLM) or AI agent to reliably perform a task. Good prompts help the AI understand:

- what role it should assume,
- what specific task it must perform,
- and how the output should be structured. (Medium)

In the context of AI-assisted penetration testing, poorly worded prompts can cause ambiguous results, misinterpretations, or even responses that suggest unsafe actions. Clear prompts reduce these risks.

Section 2 — Prompt Template: Reconnaissance

Reconnaissance is the first phase of a penetration test. It involves discovering attack surface, input vectors, and data points that could lead to vulnerabilities.

Recon Prompt Template

Goal: Guide the AI agent to methodically discover and enumerate observable interfaces or parameters without executing destructive actions.

```
Role: You are an expert penetration testing recon specialist.

Context: Target application is [target URL or IP].

Objectives:
1. Identify all visible input vectors (e.g., query parameters, form fields).
2. Enumerate reachable resources and responses.
3. Suggest non-destructive enumeration tools or techniques.

Output format:
{
  "recon_steps": [
    {"step_description": "...", "tool_or_approach": "...", "expected_evidence": "..."},
    ...
  ]
}
```

Explanation

- **Role assignment** tells the model to act as a specialist. (Ostorlab)
- **Objectives** are broken down into explicit tasks.
- **Output format** ensures the agent returns structured JSON with specific fields rather than vague prose.

Example Prompt for Recon

```
Role: Penetration recon specialist.
Target: http://testphp.vulnweb.com
Objectives:
1. List query parameters and endpoints.
2. Identify input fields and suggested passive enumeration tools.
3. Avoid active exploitation.

Output format: JSON with fields recon_steps and recommended tools.
```

Section 3 — Prompt Template: Hypothesis Formulation

In penetration testing, a **hypothesis** is a testable assumption about potential vulnerabilities or weaknesses based on observed behavior or data. Hypothesis prompts help the agent propose targeted tests and expected evidence.

Hypothesis Prompt Template

```
Role: You are an experienced penetration tester focusing on vulnerability hypotheses.

Context: Target: [target details].

Hypothesis:
```

```
- Identify one assumption about a potential weakness (e.g., parameter X may allow SQL injection).

Test Plan:
- Describe a safe plan to test that hypothesis.
- List tools or commands.
- Define expected evidence that confirms or rejects the hypothesis.

Output format:
{
  "hypothesis_statement": "...",
  "test_plan": [
    {"action": "...", "tool": "...", "expected_result": "..."},
    ...
  ]
}
```

Example Prompt for Hypothesis

```
Role: Experienced pentester.
Context: http://testphp.vulnweb.com/listproducts.php?cat=1
Hypothesis:
- The 'cat' parameter may be susceptible to SQL injection.

Test Plan:
- Describe non-destructive tests first (e.g., response comparisons).
- Include safe payload examples.
- Define expected behavior if injection is present.

Output format: JSON.
```

Section 4 — Prompt Template: Reporting

At the end of a workflow, the agent should generate a clear, structured **report** of findings, including evidence collected and recommendations.

Report Prompt Template

```
Role: Expert penetration tester and report author.

Context: Based on prior recon and hypothesis tests.

Report:
{
  "summary": "...",
  "findings": [
    {
      "description": "...",
      "evidence": "...",
      "confidence": "...",
      "recommendation": "..."
    }
  ],
  "tools_used": [...],
  "next_steps": "..."
}
```

Example Prompt for Report

```
Role: Security analyst.
Based on previous steps on http://testphp.vulnweb.com
Produce a JSON report with:
- a summary of what was tested,
```

```
- any vulnerabilities found,
- evidence explaining why,
- actionable recommendations.
```

Section 5 — Avoiding Open-Ended or Destructive Instructions

Security testing — especially when guided by AI — must be conducted safely and ethically. Open-ended instructions can cause an AI to generate unsafe actions or misinterpret objectives. Examples of **unsafe prompts to avoid**:

- “Hack all vulnerabilities on this server.”
- “Run all exploits you can find.”
- “Cause the application to crash.”

These prompts lack scope, constraints, and ethical awareness. They do not clearly define **authorized scope** or **measurement outcomes**, and could imply destructive intent.

Instead, always constrain tasks with:

1. **Explicitly stated authorization** (e.g., “testing only on the authorized lab site”).
2. **Clear action boundaries** (e.g., “passive enumeration only”).
3. **Structured output formats** (JSON with specific fields).

Section 6 — Prompt Safety Considerations

In addition to avoiding destructive phrasing, you must be aware of adversarial prompt risks such as **prompt injection** — where model instructions are overridden or manipulated based on malicious content. Prompt injection is a known security class affecting generative AI systems. It arises when the model can’t distinguish system instructions from manipulated user inputs. (Wikipedia)

Mitigations include:

- sanitizing inputs before passing them into prompts,
- enclosing roles and instructions separately from user-provided data,
- using model features that distinguish system vs user prompts where available.

These safety measures help ensure agent outputs remain aligned with intended guidance and avoid unintended execution beyond scope.

Section 7 — Summary of Best Practices

Goal	Best Practice
Guide agent to specific tasks	Use structured role, objective, and output format
Test hypotheses safely	Include planned tests with expected evidence fields
Compile findings clearly	Use JSON report templates
Prevent destructive outputs	Constrain with ethical boundaries
Avoid adversarial instruction manipulation	Sanitize inputs and enforce prompt structure

Memory, Context, and Workspace Boundaries

1 — Memory and Context in AI Agents

What Memory Means for AI Agents

AI agents often operate in workflows that span multiple steps and require information from prior interactions. There are two general classes of memory:

- **Short-term / in-context memory** — the conversation history and immediate state that a model uses to generate a response right now.

- **Long-term / out-of-context memory** — external storage (e.g., a database, vector store, or knowledge graph) that persists information across sessions or workflows. (Documentação LangChain)

As the history grows, agents can become overloaded with context, degrade in performance, or exceed token limits of language models. (Maxim AI)

Memory Types and Use

Many systems also differentiate *episodic memory* (specific past actions, decisions) and *semantic memory* (general factual knowledge). Hybrid memory architectures combine types to balance relevance and storage cost. (LinkedIn)

---

2 — Managing Tool Memory and Conversation History Limits

Why Limits Matter

Language models have a **context window size** — the maximum amount of information they can consider for generating an output. Including too much history:

- uses more tokens (higher cost)
- slows responses
- may confuse the model with outdated or irrelevant context
- risks dropping key facts when over limits (Maxim AI)

Typical Strategies

1. **Sliding Window (Recent History Buffer)**
   Keep the most recent N turns or tokens intact and drop older messages beyond the limit. This preserves _immediate_context but may lose earlier details. (Medium)
2. **Rolling Summaries**
   Create compressed summaries of earlier context and retain them, letting detailed, verbatim history be replaced by a distilled

representation of past interactions. (Medium)

3. **Entity Ledger / Structured Context**
   Extract key structured entities (e.g., hostnames, vulnerabilities found, commands used) and store these separately. Instead of including raw conversation text, feed structured facts into context. (Medium)
4. **Hybrid Memory Architectures**
   Combine recent raw history for immediate relevance with semantic summaries and structured facts for overall continuity. (diamantai.substack.com)

---

3 — When to Truncate vs Preserve Context for Reproducibility

Truncate When:

- The conversation has **irrelevant or outdated details**.
- You need to **stay under model token limits**.
- The session has evolved past earlier context that no longer guides current actions.
  Example: dropping an old error message once it's been resolved.

Preserve When:

- Early decisions affect later steps (e.g., the first reconnaissance findings on a target).
- You want **reproducibility** — the ability to rerun or audit workflows and results reliably.
- You need a *full trace* of decisions and actions for reporting or compliance.

Balance With Summarization

Rather than a binary keep/drop choice, generate **summaries** of older context before truncation. This retains critical information while conserving tokens. (Medium)

---

4 — Reproducibility and Versioning of Context

For pentesting or security analysis, reproducibility means you can recreate the same agent behavior later. Best practices include:

- **Versioned Memory Snapshots**: Save checkpoints or summaries after each major workflow stage.
- **Explicit Prompt Recording**: Save the exact prompts used, including metadata like timestamps and tool outputs.
- **Immutable Logs**: Store logs in write-once systems to prevent accidental modification.
  These techniques help maintain an audit trail and enable others to understand how conclusions were reached. (Migalhas)

---

5 — Labeling and Sanitizing Logs Before Storage or Sharing

Why It Matters

Logs often contain sensitive information (e.g., internal IPs, credentials, proprietary URLs). Before saving or sharing logs:

Label Logs Clearly

- **Context** (what workflow step)
- **Timestamp**
- **Agent/Tool name**
- **Input and output fields**

Consistent labeling helps organize data and supports reproducibility. (Migalhas)

Sanitize Sensitive Data

Remove or mask any:

- authentication tokens
- API keys

- passwords
- personal data

Sanitization ensures logs are safe for storage or reuse, especially if shared with collaborators or QA systems.

Techniques for Sanitization

- **Regex filtering** to mask patterns (e.g., all tokens replaced with `<REDACTED>`).
- **Context redaction** if it contains irrelevant but sensitive details.
- **Structured masking** using policies that define which log fields can be persisted.

Sanitized logs improve security without undermining reproducibility.

---

6 — Practical Example: Context Management in a Pentest Workflow

Imagine an AI agent performing reconnaissance on a controlled target like `http://testphp.vulnweb.com`:

1. **Initial Recon Step**
   - Raw HTTP responses collected.
   - Store immediate context (e.g., visible parameters).
   - Summarize key observations (e.g., “Parameter `cat` appears injectable”).
   - Save snapshot before moving on.
2. **Subsequent Hypothesis Test**
   - Only recent context plus the summarized key observations are fed to the model.
   - Old raw responses get condensed to structured facts.
3. **Reporting**
   - Logs used in reports include clear labels and sanitized outputs (no raw sensitive test data beyond the lab environment).

By combining summaries and structured memory, the agent stays within model limits while preserving traceable decision context. (Medium)

7 — Summary Best Practices

Task	Best Practice
Manage conversation history	Use sliding windows + summaries
Preserve context for reproducibility	Create structured records & snapshots
Truncate safely	Summarize then drop, not blind delete
Label logs	Include metadata: timestamp, agent, step
Sanitize logs	Remove credentials and sensitive data

Interpreting Fingerprints and Discovery Output

1. Understanding Fingerprinting in Pentesting

Fingerprinting is the process of **identifying the technologies, frameworks, behaviors, and structural characteristics** of a target application.
It gives you the foundational map needed before any hypothesis testing.

Fingerprinting typically answers:

- What web server is used? (Apache, nginx, IIS)
- What programming language is running? (PHP, ASP.NET, Node)
- What CMS or frameworks exist?
- What routes, parameters, forms, cookies, or headers appear?
- What predictable patterns or anomalies show vulnerability opportunities?

Discovery output is the **raw material** your agent must interpret to move into structured testing.

2. Fingerprinting the Lab Target: testphp.vulnweb.com

When performing reconnaissance on `http://testphp.vulnweb.com/`, typical initial indicators include:

2.1 Response Headers

A GET request will often reveal:

- Server: Apache
- X-Powered-By: PHP
- Content-Type: text/html
- Cookies being set (like PHPSESSID)

This instantly tells you:

- The backend is PHP
- Likely running on a Linux/Apache stack
- Potentially vulnerable to classic PHP misconfigurations (LFI/RFI, SQLi, parameter pollution)

2.2 URL Structure and Parameters

You will immediately see:

- `/listproducts.php?cat=1`
- `/product.php?pic=...`
- `/userinfo.php?user=...`

The presence of **numeric parameters**, unencoded strings, or predictable names is a strong indicator of:

- SQL Injection potential
- Reflection or stored injection vectors
- Inclusion vulnerabilities

- Weak parameter validation

2.3 Metadata From HTML Source

Viewing source identifies:

- Forms lacking CSRF tokens
- Hidden fields
- Direct file references
- Input names used in GET or POST requests

You may also spot references to:

- `/admin/`
- `/images/`
- `/js/`
- `/search.php`

Each is a recon pivot point.

2.4 Directory and File Discovery

The server exposes typical PHP entrypoints, which helps classify:

- Potential outdated scripts
- Likely database interaction locations
- Parameter-rich endpoints prone to injection

This fingerprinting defines the **attack surface shape** before any vulnerability hypothesis.

---

3. How to Interpret Discovery Outputs

Once you run discovery tools like:

- curl
- dirsearch
- gobuster
- parameter mining tools
- passive crawlers

you will collect fingerprints that reveal:

3.1 Technology Clues

Example fingerprint:

```
Server: Apache/2.4.x
X-Powered-By: PHP/5.x
```

Interpretation:

- Old PHP version → outdated functions → possible SQLi patterns
- Apache default configs often leak directory structure or allow predictable URL enumeration

3.2 Parameter Fingerprints

Example discovery:

```
Found parameter: cat=1
Found parameter: pic=../images/120.jpg
Found parameter: user=3
```

Interpretation:

- Numeric parameters → test for SQLi
- File name parameters → test for LFI or path traversal
- User ID parameters → test for IDOR
- Repeated patterns → automated enumeration feasible

3.3 Behavioral Fingerprints

If a slight modification triggers a change in behavior:

```
/listproducts.php?cat=1'
→ returns SQL error
```

That means:

- The application leaks internal DB errors
- SQLi hypothesis becomes high probability
- DBMS type can be inferred from the error
- Output-based injection is likely possible

3.4 Authentication and Session Fingerprints

Example:

```
Set-Cookie: PHPSESSID=abcd1234
```

Interpretation:

- Standard PHP session handling
- Session fixation or predictable session behavior may exist

3.5 File System Fingerprints

Discovery of:

```
/images/
/admin/
/private/
```

Interpretation:

- `/admin/` may contain weak login
- `/images/` helps confirm directory traversal risks
- Static folders often leak naming conventions

---

4. Recon Prompts for AI Agents (Deep and Structured)

Below are **ready-to-use**, deeply structured prompts designed to guide AI agents in interpreting fingerprints and planning follow-up recon. You can paste them directly into an AI system.

4.1 Prompt — Fingerprint Interpretation Recon

```
You are a penetration testing reconnaissance analyst.

Target: http://testphp.vulnweb.com/

Task:
1. Review discovered URLs, parameters, headers, cookies, and HTML metadata.
```

```
2. Interpret what each fingerprint implies about:
   - backend technologies
   - likely vulnerabilities
   - parameter behaviors
   - file system exposure
3. Produce a structured fingerprint map that includes:
   - Technology stack inference
   - Parameter classification (numeric, file references, IDs, search terms)
   - Behavioral observations
   - Potential vulnerability hypotheses supported by evidence

Output as structured analysis, not a tool command.
```

4.2 Prompt — Recon Discovery Expansion

```
You are conducting the discovery phase of a web penetration test.

Target: http://testphp.vulnweb.com/

Objective:
1. Enumerate all reachable endpoints and parameters.
2. For each discovery:
   - classify parameter type (ID, file, numeric, string)
   - identify patterns that commonly indicate vulnerabilities
   - map possible attack surfaces
3. Create a prioritized list of recon pivots.

Avoid exploitation. Focus only on discovery interpretation.
```

4.3 Prompt — Behavior-Based Fingerprint Interpretation

```
Role: Fingerprint behavior analyst.

Given the URL patterns and responses from http://testphp.vulnweb.com/,
perform a behavior-oriented interpretation:

1. Identify differences in server behavior when parameters are modified.
2. Infer database behavior, file inclusion patterns, or reflection patterns.
3. Describe probable vulnerability classes supported by fingerprint evidence.
4. Organize findings into an annotated discovery matrix.
```

```
Gemini - gemini-cli
Session  Actions  Edit  View  Help
> Role: Fingerprint behavior analyst.

  Given the URL patterns and responses from http://testphp.vulnweb.com/,
  perform a behavior-oriented interpretation:

  1. Identify differences in server behavior when parameters are modified.
  2. Infer database behavior, file inclusion patterns, or reflection patterns.
  3. Describe probable vulnerability classes supported by fingerprint evidence.
  4. Organize findings into an annotated discovery matrix.

✦ Okay, I will now perform a behavioral analysis of http://testphp.vulnweb.com/ to identify potential
  vulnerabilities. I will probe the server with various inputs, analyze the responses, and present the findings
  in an annotated discovery matrix.

  First, I will explore the application to understand its structure and identify potential areas of interest for
   testing. I'll start by listing the files in the current directory to see if there's any existing information,
   and then I'll fetch the main page of the website.

  √  Shell ls -F (List the files in the current directory to check for any existing information before star…

     gemini.js*
     vulnerability_report.md

✦ I see the files gemini.js and vulnerability_report.md. I will proceed with fetching the main page of
  http://testphp.vulnweb.com/.

  ⊸  WebFetch Processing URLs and instructions from prompt: "Please fetch the content of the URL http://tes…

⠼ Investigating Website Structure (esc to cancel, 10s)
```

You can save the prompt to a .md file and have the agent read the file.

---

5. Connecting Fingerprints to Next Recon Steps

Proper fingerprint interpretation guides what happens next:

From Numeric Parameters → SQLi Recon

- Try adding `'`, `"`, `--`, `OR 1=1`
- Compare responses
- Look for SQL error leakage

From File Parameters → LFI/LFD Recon

- Try adding `../` patterns
- Try alternate file extensions
- Observe for directory traversal patterns

From User Parameters → IDOR Testing

- Replace numeric identifiers
- Observe changes in returned content

From HTML Source Clues → Form Enumeration

- Extract forms
- Identify input types
- Map POST parameters
- Review hidden fields

---

6. Building a Fingerprint Map for testphp.vulnweb.com

A complete mapping may look like:

6.1 Technology Stack

- Apache server
- PHP backend
- Likely MySQL database

6.2 Identified Endpoints

- `/listproducts.php?cat=`
- `/product.php?pic=`
- `/search.php?query=`
- `/userinfo.php?user=`

6.3 Parameter Classification

Parameter	Type	Interpretation
cat	Numeric	SQLi candidate
pic	File/Path	LFI candidate
user	Numeric ID	IDOR candidate
query	Search term	XSS candidate

6.4 Behavioral Findings

- Error messages leak SQL syntax failures
- Reflection occurs in search fields
- File parameters do not sanitize traversal sequences

---

7. Using Fingerprint Interpretation to Form Hypotheses

With the above information, we can generate:

Hypothesis Example 1

The `cat` parameter may be vulnerable to SQL injection because the application returns SQL syntax errors when the value includes a quote.

Hypothesis Example 2

The `pic` parameter may allow local file inclusion because it accepts relative filenames and returns different errors when path patterns are modified.

Hypothesis Example 3

The `user` parameter may expose data belonging to other users, indicating missing access control validation.

---

8. Summary

This lesson provided:

- A deep explanation of fingerprinting and discovery interpretation
- How to analyze fingerprints systematically
- Technology, behavior, and parameter inference techniques
- Recon prompts for AI agents to interpret discovery data
- A complete fingerprint map of `testphp.vulnweb.com`
- How fingerprint interpretation leads to vulnerability hypotheses

https://www.youtube.com/watch?v=WIPJUv7wka8&t=8s

Building and Analyzing an Endpoint Inventory

1. The Role of Endpoint Inventory in Pentesting

Before vulnerability discovery, an AI agent must understand:

- what routes exist
- how they behave
- which parameters they expose
- which endpoints require authentication
- what attack surface overlaps exist

An endpoint inventory serves as the **map of the application**, ensuring recon and testing stay systematic instead of random.

---

2. Techniques to Extract Endpoints

You can collect endpoints from **static analysis**, **traffic observation**, and **active/passive crawling**. The most common techniques are:

2.1 HTML Parsing & DOM Enumeration

Manually or automatically extract endpoints by inspecting:

- `<a href="">`
- `<form action="">`
- `<script src="">`
- `<img src="">`
- `<link href="">`
- XMLHttpRequests or fetch calls
- Comments in HTML

Example findings in our lab:

- `/listproducts.php`
- `/product.php`
- `/userinfo.php`
- `/search.php`

These give you entry points for parameter enumeration.

2.2 Crawling Tools and Spidering

Tools like:

- OWASP ZAP spider
- Burp Suite crawler
- gospider
- hakrawler
- custom scripts

can dynamically discover endpoints by crawling and executing JavaScript.

Output often includes:

- dynamic routes
- query parameters
- hidden or nested pages

In the lab, crawlers will reveal endpoints such as:

- `/admin/`

- `/images/`
- `/comments.php`

even if not linked from the homepage.

2.3 Analyzing Network Traffic

Capturing traffic with:

- Browser DevTools (Network tab)
- Burp Proxy
- mitmproxy

helps detect endpoints triggered by:

- form submissions
- AJAX calls
- hidden API calls
- redirections
- authentication flows

Even an old PHP site like vulnweb discloses interactions such as:

- POST to `/login.php`
- GET `/logout.php`
- Hidden file references

Traffic analysis is essential for endpoints that crawlers might miss.

2.4 Sitemap (robots.txt, sitemap.xml)

Many websites publish navigational metadata:

- `/robots.txt`
- `/sitemap.xml`

While vulnweb doesn't expose full sitemaps, some sites reveal:

- admin panels
- deprecated endpoints
- high-privilege routes

which are key to expanding recon intelligently.

2.5 Parameter Mining Tools

Tools such as:

- ParamSpider
- Arjun
- Gau (GetAllUrls)
- Wayback Machine mining

help find:

- legacy parameters
- hidden parameters
- deprecated endpoints
- parameters from old site versions

Useful especially when testing large, real-world environments, but also helpful for structured methodology in the lab.

---

3. Classifying Endpoints

After extracting endpoints, classification allows the AI to reason effectively about **risk** and **access requirements**.

We use four primary classes:

3.1 Public Endpoints

Accessible without authentication.

Examples from vulnweb:

- `/`
- `/listproducts.php?cat=`
- `/product.php?pic=`
- `/search.php?query=`

These make up the **bulk of initial attack surface**.

Characteristics:

- Visible to crawlers
- Often parameterized
- Most likely to contain injection points

3.2 Authenticated Endpoints

Require login or a session cookie.

In vulnweb, examples include:

- `/userinfo.php?user=`
- `/logout.php`
- `/guestbook.php` (after posting)

Characteristics:

- Often trust user input more
- Contain sensitive logic
- Ideal for IDOR or privilege escalation testing

3.3 Admin Endpoints

Privileged routes not visible to normal users.

Vulnweb might reveal paths like:

- `/admin/`
- `/private/`
- `/admin/index.php`

Admin endpoints are **high-value targets** because:

- They often lack strong access control
- They expose sensitive configurations

- They reveal application logic

3.4 API Endpoints

Even older sites may contain simple API-like handlers.

API endpoints typically:

- return JSON or XML
- accept POST/GET parameters
- lack UI exposure

Examples in modern apps:

- `/api/login`
- `/api/search`
- `/api/user/3`

The distinction helps an agent choose appropriate payloads and testing methods.

---

4. Generating a Canonical Endpoint List

A canonical list is:

- deduplicated
- normalized
- parameter-aware
- grouped by classification

- ready for hypothesis generation

It allows AI agents to reason cleanly without noise.

Below is a canonical endpoint inventory for http://testphp.vulnweb.com/.

---

5. Canonical Endpoint Inventory for testphp.vulnweb.com

5.1 Public Endpoints

```
GET /
GET /listproducts.php?cat=
GET /product.php?pic=
GET /search.php?query=
GET /comments.php
GET /guestbook.php
```

5.2 Authenticated Endpoints

```
GET /userinfo.php?user=
POST /login.php
GET /logout.php
POST /guestbook.php
```

5.3 Admin Endpoints

```
GET /admin/
GET /admin/index.php
GET /private/
```

5.4 Static Asset Endpoints

```
/images/
/js/
/css/
```

5.5 Potential API-like Behaviors

```
/userinfo.php?user=ID
/search.php?query=TERM
```

API-like classification is about *behavior*, not presence of `/api/`.

---

6. How the AI Should Reason About the Inventory

Once the canonical list is built, the agent can:

6.1 Prioritize Recon Testing

- numeric parameters → SQLi
- file parameters → LFI
- search parameters → XSS
- user identifiers → IDOR

6.2 Map Attack Chains

Example chain in vulnweb:

1. Public SQLi on `/listproducts.php`

2. Extract user IDs
3. Attempt access on `/userinfo.php?user=`
4. Check privilege boundaries

6.3 Evaluate Access-Control Boundaries

Admin routes can be cross-referenced with:

- public access attempts
- cookie/session presence
- parameter tampering

6.4 Support Hypothesis Generation

Each endpoint becomes a hypothesis seed:

```
Hypothesis: /product.php?pic= may accept file path traversal.
Hypothesis: /search.php?query= may reflect input → XSS risk.
Hypothesis: /userinfo.php?user= may expose other users' data.
```

---

7. Recon Prompts for Endpoint Inventory

Here are fully structured prompts for AI agents to build and analyze endpoint inventories.

7.1 Prompt — Endpoint Extraction and Inventory Building

```
You are a reconnaissance analyst.

Target: http://testphp.vulnweb.com/

Task:
1. Extract endpoints by analyzing:
   - links
   - forms
   - scripts
   - network requests
2. Expand the list using parameter discovery.
3. Deduplicate and normalize the findings.
4. Classify endpoints into:
   - public
   - authenticated
   - admin
   - API-like
5. Output a canonical endpoint inventory suitable for reasoning.
```

7.2 Prompt — Endpoint Classification and Reasoning

```
You are classifying endpoints for attack surface reasoning.

Target: http://testphp.vulnweb.com/

Task:
1. Take the list of discovered endpoints.
2. Classify them:
   - public
   - authenticated
```

```
   - admin
   - static assets
   - API-like
3. Explain the logic used for each classification.
4. Produce a structured map the AI can use for further hypotheses.
```

7.3 Prompt — Attack Surface Modeling Based on Endpoint Inventory

```
You are modeling the attack surface based on endpoint inventory.

Target: http://testphp.vulnweb.com/

Using the canonical list:
1. Identify attack vectors associated with each endpoint type.
2. Prioritize endpoints based on risk signals.
3. Produce a reasoning-ready representation for hypothesis testing.
```

Conclusion

By the end of this lesson, you learned:

- How to extract endpoints from traffic, crawling, and metadata
- How to classify endpoints into meaningful categories
- How to build canonical inventories AI agents can reason about
- How to link endpoints to attack surface and hypothesis generation

Attack Surface Maps and Management

1. Understanding Attack Surface Maps

An **attack surface** is the sum of all entry points, behaviors, parameters, and interactions exposed by an application that an attacker (or pentest agent) can manipulate.

An **attack surface map** organizes this information into:

- endpoints
- parameter types
- trust boundaries
- authentication boundaries
- data flows
- privilege levels
- backend components

The map transforms fragmented recon data into a **structured model** that supports:

- hypothesis creation
- risk scoring
- prioritization
- chain-of-attack planning

It is the *blueprint* from which systematic pentesting flows.

---

2. Components of an Attack Surface Map

To build a complete map, combine the following layers:

2.1 Endpoint Layer

Every reachable route from your canonical inventory:

- `/listproducts.php?cat=`
- `/product.php?pic=`
- `/userinfo.php?user=`
- `/search.php?query=`
- `/admin/`

2.2 Parameter Layer

Define the parameter types:

- numeric → SQLi / insufficient validation
- file/path → LFI / traversal
- string → XSS / reflection
- IDs → IDOR / access control

Example for vulnweb:

Endpoint	Parameter	Type	Implication
listproducts.php	cat	numeric	SQLi
product.php	pic	file	LFI
userinfo.php	user	numeric ID	IDOR
search.php	query	string	XSS

2.3 Authentication & Trust Boundaries

Define which endpoints belong to:

- **public space**
- **authenticated space**
- **admin space**

This models where authorization bypass, privilege escalation, and horizontal/vertical access issues may occur.

2.4 Backend and Technology Surface

Map fingerprints such as:

- Apache / PHP stack
- PHP session cookies
- SQL error behavior
- Static file structure

These guide the testing strategies.

2.5 Attack Chains

Attack chains describe how one weakness leads to another:

Example chain for vulnweb:

1. SQLi in `/listproducts.php`
2. Extract usernames or IDs
3. Access `/userinfo.php?user=` to enumerate profiles
4. Test for IDOR and privilege escalation opportunities

Attack chains show **multi-step exploitation paths**, which are essential for realistic simulation.

---

3. Building an Attack Surface Map for testphp.vulnweb.com

Below is a detailed attack surface map based on the combined lessons so far.

3.1 Public Surface

```
/listproducts.php?cat=
/product.php?pic=
/search.php?query=
/comments.php
/guestbook.php
```

Risk Tendencies:

- `cat` → SQL injection
- `pic` → LFI / directory traversal
- `query` → reflected XSS
- comment/guestbook → stored XSS

3.2 User-Level Authenticated Surface

```
/userinfo.php?user=
/login.php (POST)
/logout.php
```

Risk Tendencies:

- `userinfo.php?user=` → IDOR
- login → brute-force, weak authentication, error-based information exposure
- logout → session lifecycle validation

3.3 Admin Surface

```
/admin/
/private/
```

Risk Tendencies:

- missing authentication
- directory listing exposure
- outdated admin tools
- possibility of broken access control

3.4 Application Technology Surface

Application Stack Indicators:

- Apache server
- PHP runtime
- Likely MySQL backend
- Standard PHP session management
- Static directories (`/images/` , `/css/` , `/js/`)

Attack Implications:

- Dirty error reporting → DB fingerprinting
- Outdated PHP → common vulnerability patterns
- Static structure → potential path traversal
- Unescaped reflection → XSS likelihood

3.5 Structured Attack Surface Graph

Nodes

- Endpoints
- Parameters
- Authentication zones
- Technology indicators
- Data inputs & outputs

Edges

- “This endpoint accepts this parameter”
- “This parameter influences this backend component”
- “This endpoint belongs to this trust zone”
- “This pattern suggests this class of vulnerability”

Example representation:

```
/listproducts.php
   └── cat (numeric param)
           └── influences SQL query
```

```
                    └── risk: SQL injection

/product.php
   └── pic (file param)
           └── influences filesystem read path
                   └── risk: LFI, traversal

/userinfo.php
   └── user (ID param)
           └── influences access-controlled data
                   └── risk: IDOR

/search.php
   └── query (string)
           └── reflected to page
                   └── risk: XSS
```

This graph helps an AI agent reason about *logic relationships*.

---

4. Managing the Attack Surface

Attack surface management requires:

4.1 Deduplication & Normalization

Collapse endpoints with variations into a single conceptual node:

- `/search.php?query=a`
- `/search.php?query=test`

- `/search.php?query=<input>`

All map to:

Endpoint: `/search.php`
Parameter: `query`
Type: string

4.2 Prioritization Strategies

Agents should prioritize based on:

High-value parameter patterns

- numeric → injection
- file → traversal
- user identifiers → access control flaws

High-privilege zones

Admin routes automatically rank high.

Behavioral anomalies

If an endpoint leaks server errors → test early.

Chain potential

Endpoints that bridge into deeper surfaces get priority.

4.3 Surface Reduction and Segmentation

In larger applications, focus should be segmented into:

- public surface
- authenticated surface
- privileged admin surface
- API surface

This segmentation prevents the model from reasoning about all endpoints simultaneously and improves focus and accuracy.

---

5. Prompts for Generating Attack Surface Maps

These are fully structured prompts designed for an AI agent to build or update an attack surface map.

5.1 Prompt — Build an Attack Surface Map

```
You are building an attack surface map for the target: http://testphp.vulnweb.com/

Task:
1. Combine endpoint inventory, parameter types, and fingerprint data.
2. Map each endpoint to:
   - parameter type
   - authentication level
   - backend behavior implied
3. Build a hierarchical attack surface map showing:
   - public, authenticated, admin surfaces
```

```
   - attack vectors associated with each
4. Include connections that indicate multi-step attack chains.
```

5.2 Prompt — Update Attack Surface After New Recon Findings

```
You are updating the attack surface model.

Given new endpoints or parameters:
1. Insert them into the attack surface map.
2. Reclassify if needed (public/auth/admin/API-like).
3. Adjust risk prioritization.
4. Identify new attack chain possibilities.
```

5.3 Prompt — Attack Surface Prioritization Planning

```
You are prioritizing attack surfaces for testing.

Target: http://testphp.vulnweb.com/

Task:
1. Evaluate each endpoint class.
2. Rank attack vectors by:
   - exploitability
   - potential impact
   - multi-chain leverage
3. Produce a prioritized testing plan based on the map.
```

6. How an AI Agent Uses an Attack Surface Map

An AI pentest agent uses the map to:

- create hypotheses
- plan safe tests
- avoid redundant or invalid tests
- trace reasoning
- build reproducible pentest flows

The map ensures the agent thinks like a structured pentester rather than hallucinating possible targets.

Conclusion

This lesson covered:

- What attack surface maps are
- How to build them from recon and endpoint inventories
- How to classify and connect surfaces
- How to prioritize and manage the attack surface
- Prompts for generating and updating maps
- A complete attack surface map for testphp.vulnweb.com

Recon Beyond the Basics: Advanced Discovery for Pentesting and Bug Bounty

1. Recon as an Intelligence Operation

In bug bounty and pentesting, **reconnaissance is intelligence collection**.
You're not testing vulnerabilities yet—you're creating a *complete intelligence picture* of the target.

Good recon answers:

- What assets exist?
- What technologies power them?
- Where does user input enter the system?
- How does the system behave in edge cases?
- What routes are hidden, deprecated, or unlinked?
- How can these surfaces be exploited later?

Professional bounty hunters often say:

> "The person with the best recon wins."

Because vulnerability discovery becomes trivial when recon uncovers **non-obvious elements** other researchers miss.

---

2. Multi-Angle Recon Strategies

Recon is not a single action—it is a composite of several *modes of analysis*:

2.1 Structural Recon

Identify the underlying structure:

- folders
- routes

- parameters
- static assets
- admin paths
- backup files

In the vulnweb lab, structural recon reveals:

- `/admin/`
- `/private/`
- `/listproducts.php`
- `/product.php`
- `/guestbook.php`
- `/userinfo.php`

These become pivots into deeper recon.

2.2 Parametric Recon

Cataloging every input parameter and its type.

Examples from vulnweb:

Parameter	Endpoint	Meaning	Type
cat	listproducts.php	category	numeric
pic	product.php	image path	file/path
user	userinfo.php	user ID	numeric ID
query	search.php	search term	string

Parametric recon enables:

- SQLi opportunities
- XSS locations
- directory traversal
- IDOR and privilege escalation

2.3 Behavioral Recon

Behavioral recon focuses on **how the application responds**, not just what endpoints exist.

For example:

- If adding a `'` causes SQL errors → DB leakage
- If modifying an ID changes the result → access control gaps
- If `%2e%2e/` returns an error → file inclusion vectors
- If missing parameters break pages → weak validation

Behavioral recon provides evidence of internal logic.

2.4 Asset Recon

Looking beyond the main application:

- subdomains
- staging environments
- forgotten pages

- development tools
- misconfigured admin panels
- backup files like `.zip` or `.bak`

In bug bounty, *asset discovery* often leads to:

- old versions
- exposed test environments
- misconfigured features

These are high-value intel nodes.

2.5 Passive Recon

Collect data without touching the server:

- Google dorking
- Archive.org snapshots
- GitHub leaks
- public JS files revealing internal routes
- fuzzing parameter names from comments or scripts
- fingerprinting frameworks via headers or JS bundles

Passive recon is safe, scalable, and often yields insights other hunters miss.

---

3. Recon Applied to testphp.vulnweb.com — A Deep Scan Walkthrough

Let’s apply multi-angle recon on our controlled lab.

3.1 Endpoint and Parameter Enumeration

Tools like crawlers, DevTools, or Burp Proxy reveal:

```
/listproducts.php?cat=
/product.php?pic=
/search.php?query=
/userinfo.php?user=
/admin/
/private/
/guestbook.php
```

3.2 Identify Input Vectors

The parameter map:

- `cat` → SQL-injection candidate
- `pic` → file path candidate
- `user` → IDOR candidate
- `query` → XSS candidate
- form submissions → stored XSS candidate

3.3 Detect Hidden Behavior

By altering parameters:

- adding `'` causes SQL errors → misconfigured database error handling
- changing `user` reveals new profiles → weak access control
- injecting `<script>` into search fields returns reflections → XSS potential

Behavioral recon confirms how the backend handles malformed input.

3.4 Identify High-Value Recon Targets

From a bug bounty perspective, the most interesting surfaces are:

- admin directories
- authenticated profile endpoints
- file-based parameters
- reflected search fields
- endpoints that interact with databases

Each becomes a building block for deeper enumeration.

---

4. Recon Techniques Used by Top Bug Bounty Hunters

Elite bounty hunters don’t rely on only one method—they combine signals.

Below are **top-level techniques** used in real programs.

4.1 Asset Expansion Recon

You find assets that the company didn’t list explicitly.

This includes:

- dev/test/staging versions
- mobile endpoints
- orphaned APIs
- forgotten admin tools

Often found via:

- JavaScript file analysis
- subdomain enumeration
- path fuzzing
- historical snapshots

4.2 Stack Fingerprinting Recon

Understanding the tech stack helps predict vulnerabilities.

For vulnweb:

- PHP + Apache → SQLi, LFI, RFI
- Old PHP functions → weak sanitization
- predictable directories → directory traversal
- leaked DB errors → poor hardening

Modern stacks reveal patterns too, like:

- Node.js → NoSQL injection
- Java → deserialization
- Laravel → debug endpoints

4.3 Behavioral Recon (Deep)

Try altering parameters in:

- encoding (`%00` , `%2f` , `%25`)
- injection (`"` , `'` , `--`)
- type coercion (`true` , `null`)
- oversized values
- unexpected formats
- Unicode edge cases

Each behavior forms a **fingerprint** of the backend.

4.4 Relationship Recon

Map how endpoints relate:

- Which endpoints call each other
- Which use the same backend function
- Which share authentication requirements
- Which form chains

This helps reveal:

- pivot points
- privilege escalation paths
- multi-step exploitation sequences

---

5. Recon Prompts for AI Agents

Below are **structured prompts** for automating advanced recon reasoning.

5.1 Prompt — Multi-Angle Recon Analysis

```
You are a deep reconnaissance analyst.

Target: http://testphp.vulnweb.com/

Perform a multi-angle recon:
1. Structural recon: endpoints, files, folders.
2. Parametric recon: classify parameters.
3. Behavioral recon: compare responses to modified inputs.
4. Asset recon: identify hidden, admin, or deprecated paths.
5. Produce a unified recon intelligence summary.
```

5.2 Prompt — Passive Recon Expansion

```
You are gathering passive recon intelligence.
```

```
Target: http://testphp.vulnweb.com/

Task:
1. Analyze public-facing assets.
2. Infer directory structures from HTML and JS.
3. Identify legacy or backup paths.
4. Produce a list of potential high-value recon targets.
```

5.3 Prompt — Bug Bounty Recon Prioritization

```
You are optimizing recon for bug bounty.

Target: http://testphp.vulnweb.com/

Task:
1. From all discovered assets, identify high-value bounty targets.
2. Classify by potential severity.
3. Map attack chains that could lead to meaningful impact.
```

---

6. Recon Intelligence Matrix (Example Output Form)

A strong recon structure looks like this:

Category	Data	Interpretation
Structural	/admin/	high-value target
Parametric	pic=	file inclusion vector

Category	Data	Interpretation
Behavioral	SQL errors	injection-prone
Asset	/private/	hidden surface
User-facing	guestbook.php	stored XSS vector
Server clues	PHP version signals	outdated stack

This gives the agent a rich foundation for future testing.

---

7. Summary

In this lesson you learned:

- Recon is an intelligence collection discipline
- The multi-angle recon strategy used by elite bug bounty hunters
- How to extract deeper intel from endpoints, parameters, and behavior
- How to analyze the target beyond simple enumeration
- How to prepare rich recon data for AI reasoning
- Buckets of recon prompts to automate analysis

This forms the backbone of *offensive intelligence operations* in both pentesting and bug bounty.

If you want another lesson such as:

- **Deep Parametric Fuzzing**
- **Recon Automation Architecture**
- **Behavioral Recon Patterns**
- **Recon for Authentication Flaws**

1. Why Map Behavior to OWASP/CWE?

During recon, you collect **behaviors**, **responses**, and **patterns**, not formal vulnerabilities.
To produce a structured report or drive automated reasoning, you must convert:

- “parameter accepts quotes”
- “endpoint leaks SQL error”
- “parameter reflects user input”

into standardized categories like:

- **OWASP: A03 – Injection**
- **OWASP: A01 – Access Control**
- **CWE-89 – SQL Injection**
- **CWE-79 – XSS**

This ensures:

- consistency across findings
- prioritization aligned with industry frameworks
- clearer communication with stakeholders
- reproducibility for AI agents and automated tooling

---

2. Technical Behaviors That Map to OWASP and CWE

Below is a complete mapping table.
Each **behavior** leads to one or more OWASP categories and CWE identifiers.

2.1 Behavior: Parameter Accepts Quotes (‘ or “) and Breaks Query

Observed Behavior

- App throws SQL errors when you add `'` or `"`.
- Altering numeric parameters changes backend query behavior.

Likely OWASP Category

- **OWASP A03:2021 – Injection**

Likely CWE

- **CWE-89 – SQL Injection**
- **CWE-20 – Improper Input Validation**

Example from Vulnweb

`/listproducts.php?cat=1'` → SQL error.

2.2 Behavior: Reflected User Input Appears in Page Output

Observed Behavior

- Injected `<script>alert(1)</script>` or `<b>test</b>` is reflected in HTML.

Likely OWASP Category

- **OWASP A03:2021 – Injection**

- **OWASP A05:2021 – Security Misconfiguration** (if debug behavior is present)

Likely CWE

- **CWE-79 – Cross-Site Scripting (XSS)**
- **CWE-116 – Improper Encoding**

Example from Vulnweb

`/search.php?query=<script>alert(1)</script>`

2.3 Behavior: Changing ID Parameter Shows Another User's Data

Observed Behavior

- `/userinfo.php?user=1` → User A
- `/userinfo.php?user=2` → User B

Likely OWASP Category

- **OWASP A01:2021 – Broken Access Control**

Likely CWE

- **CWE-639 – Authorization Bypass Through User-Controlled Keys**
- **CWE-284 – Improper Access Control**

Example from Vulnweb

User enumeration possible via numeric `user=` parameter.

2.4 Behavior: File Path Parameter Appears to Control Server File Access

Observed Behavior

- `pic=../../etc/passwd` yields different server responses
- Parameter resolves real file paths

Likely OWASP Category

- **OWASP A05:2021 – Security Misconfiguration**
- **OWASP A04:2021 – Insecure Design**

Likely CWE

- **CWE-22 – Path Traversal**
- **CWE-73 – External Control of File Path**

Example from Vulnweb

`/product.php?pic=../images/12.jpg`

2.5 Behavior: Directory Listing Exposed

Observed Behavior

- Browsing `/admin/` or `/images/` lists files.

Likely OWASP Category

- **OWASP A05:2021 – Security Misconfiguration**

Likely CWE

- **CWE-548 – Information Exposure Through Directory Listing**

Example from Vulnweb

`/admin/` shows structure.

2.6 Behavior: Session Tokens Predictable or Weak

Observed Behavior

- `PHPSESSID` looks weak or not renewed on login.
- Session fixation possible.

Likely OWASP Category

- **OWASP A07:2021 – Identification and Authentication Failures**

Likely CWE

- **CWE-384 – Session Fixation**
- **CWE-331 – Weak Random Number Generation**

2.7 Behavior: Server Reveals Stack Info in Headers or Errors

Observed Behavior

- Apache version disclosed
- PHP version disclosed
- SQL engine errors printed to user

Likely OWASP Category

- **OWASP A05:2021 – Security Misconfiguration**

Likely CWE

- **CWE-209 – Information Exposure Through Error Messages**
- **CWE-200 – Exposure of Sensitive Information**

Example from Vulnweb:

`X-Powered-By: PHP/5.x`

---

3. Mapping Recon Signals Systematically

To make this reproducible, every recon finding should be converted into:

1. **Behavioral signal**
2. **Likely root cause**
3. **Matching OWASP category**
4. **Matching CWE identifier**
5. **Evidence and payload used**

This transforms recon into structured, AI-understandable intelligence.

---

4. Recon → OWASP/CWE Decision Tree

Below is a practical decision flow used by experts and AI agents.

Step 1 — Does modifying input change backend execution?

→ Injection

- SQLi → CWE-89
- XSS → CWE-79
- Command injection → CWE-77

Step 2 — Does ID or object reference expose another user's data?

→ Broken Access Control

- CWE-639
- CWE-284

Step 3 — Does endpoint load files based on user input?

→ File inclusion

- CWE-22
- CWE-73

Step 4 — Do server messages leak internal components?

→ Security Misconfiguration

- CWE-209
- CWE-200

Step 5 — Do authentication or session behaviors appear flawed?

→ Identification and Authentication Failures

- CWE-384
- CWE-287

Step 6 — Does the application treat public and admin endpoints similarly?

→ Broken Access Control

- CWE-862

This decision tree handles 95% of early-stage recon.

---

5. Standardized Mapping Table for AI Agents

Below is the exact mapping format an AI agent should use.

Format

```
Technical Behavior:
Parameter Type:
Observed Evidence:
Likely OWASP Category:
Likely CWE ID:
Notes:
```

Example for Vulnweb

```
Technical Behavior:
Parameter 'cat' accepts quotes and breaks SQL query.

Observed Evidence:
'/listproducts.php?cat=1'' → SQL syntax error.

Likely OWASP:
A03:2021 — Injection

Likely CWE:
CWE-89 — SQL Injection

Notes:
High-confidence injection vector.
```

This ensures reproducibility and a standardized intelligence workflow.

---

6. Prompts for Agents to Map Behavior → OWASP/CWE

Here are structured prompts to guide AI pentest agents in mapping recon data to standardized vulnerability categories.

6.1 Prompt — Map Behavioral Findings to OWASP/CWE

```
You are analyzing recon behavior from http://testphp.vulnweb.com/.

For each technical behavior:
1. Identify the root cause.
2. Map to the most likely OWASP Top 10 category.
```

```
3. Map to the corresponding CWE ID.
4. Provide a structured explanation of why the mapping fits.
```

6.2 Prompt — Convert Raw Recon Logs to Standard Vulnerability Models

```
Given raw recon signals and endpoint behaviors:
1. Extract meaningful technical behaviors.
2. Classify them into OWASP Top 10 groups.
3. Assign CWE identifiers to each behavior.
4. Output in a standardized, reasoning-friendly JSON format.
```

6.3 Prompt — OWASP/CWE Mapping for Endpoint Inventories

```
Using the endpoint inventory for http://testphp.vulnweb.com/,
map each parameter type and observed behavior to:
- OWASP category
- CWE ID
- Risk justification
```

---

7. Summary

After this lesson, you now know how to:

- Interpret technical behaviors from recon

- Map them into OWASP and CWE classifications
- Produce standardized, structured vulnerability intelligence
- Enable AI agents to understand and categorize attack surface data
- Apply this to the testphp.vulnweb.com lab effectively

This is a crucial skill for:

- pentest reporting
- bug bounty submissions
- automated scanning
- AI-powered assessment systems

If you want, the next lesson can be:

- **“Building a Vulnerability Evidence Catalog”**
- **“Structured Recon Automation Layers for AI Agents”**
- **“From Recon to Exploitation Planning (without exploiting)”**

Just tell me which direction you prefer.

Severity, Risk Narrative and Business Impact

1. The Purpose of a Risk Narrative

A **risk narrative** is a bridge between technical data and organizational decision-making.
It explains:

- **what the vulnerability is**
- **how an attacker could practically exploit it**
- **what real harm it causes**
- **how likely it is**

- **who is affected**
- **what the business impact is**

A good narrative enables executives to say:

> "I understand the risk, the impact, and the cost of fixing it."

A bad narrative forces them to guess.

---

2. Converting Technical Behavior Into Business Impact

Below is the workflow used by professional pentesters and auditors.

2.1 Step 1 — Identify the Technical Issue

Example technical finding from Vulnweb:

```
/listproducts.php?cat=1' → returns SQL error
```

2.2 Step 2 — Link Behavior to a Concrete Exploit Path

Connect the observed behavior to what it *means*:

- A SQL error means the application dynamically constructs SQL queries.
- Parameter manipulation shows input is not sanitized.
- The database is likely exposed to injection exploitation.

2.3 Step 3 — Describe the Realistic Attacker Outcome

State what an attacker can *actually* achieve—not hypothetical extremes.

Example outcomes for SQLi:

- Extract user data
- Modify product listings
- Dump credentials
- Escalate to full database access

No speculation. Only plausible results based on recon evidence.

2.4 Step 4 — Translate Into Business Language

Executives understand:

- **Data exposure**
- **Financial fraud**
- **Customer trust erosion**
- **Compliance violations**

They do **not** connect with:

- UNION SELECT
- Blind injection
- Stack traces

Example conversion:

Technical:
“Numeric parameter `cat` is vulnerable to SQL injection.”

Business-facing:
“Attackers could extract or alter product data, customer information, or internal records stored in the database. This may result in data breaches, compliance failures, or loss of integrity in online content.”

---

3. Avoiding Generic Severity Inflation

Many reports incorrectly rate everything as High or Critical.

Realistic severity is based on the **triad**:

1. **Exploitability**
   - How easy is it to exploit?
   - Is user interaction required?
   - Is authentication required?
2. **Exposure**
   - Is the endpoint public?
   - Is the vulnerable parameter accessible without login?
3. **Impact Scope**
   - What systems or data are affected?
   - Does it enable lateral movement?

Use this scoring to avoid exaggeration.
The goal is credibility, not drama.

---

4. Example: Turning Vulnweb Findings Into Risk Narratives

Using recon data, here is how to frame risk for each class of issue.

4.1 SQL Injection — /listproducts.php?cat=

Severity (Realistic)

Critical, because:

- endpoint is **public**
- exploitability is **trivial**
- impact includes **database compromise**

Business-Facing Impact Narrative

Attackers can manipulate the product database and extract sensitive information. This leads to breach of customer data, financial exposure, integrity damage, and regulatory penalties.

Prioritized Remediation

1. Implement parameterized queries (1–2 days).
2. Add input sanitization at controller level (1 day).
3. Deploy WAF rules for injection signatures (4–6 hours).

4.2 XSS — /search.php?query=

Severity

Medium to High, depending on outcome.

Business-Facing Narrative

Attackers can inject malicious scripts into search pages, potentially stealing session cookies, performing actions on behalf of users, or defacing content. This threatens user trust and brand reputation.

Remediation

1. Encode all output that reflects user input (1 day).
2. Apply Content Security Policy (1–2 days).
3. Add server-side validation (1 day).

4.3 IDOR — /userinfo.php?user=

Severity

High, because:

- direct access to sensitive user data
- simple URL manipulation
- no authentication upgrade needed

Business-Facing Narrative

Attackers can access other users' profiles by modifying the URL. This constitutes a privacy breach and exposes the business to compliance violations (GDPR/PII requirements).

Remediation

1. Enforce server-side authorization checks (1–2 days).

2. Introduce session-aware access mapping (2 days).
3. Update logging to detect horizontal access attempts (4 hours).

4.4 File Inclusion — /product.php?pic=

Severity

High if local file inclusion is confirmed.
Medium if traversal is limited but still unstable.

Business-Facing Narrative

An attacker could force the server to load unintended files. This may reveal internal configuration, source code, or credentials, enabling deeper compromises.

Remediation

1. Restrict paths to whitelisted directories (1 day).
2. Validate filenames on the server (1 day).
3. Disable PHP file inclusions from user variables (4–8 hours).

---

5. Severity Calibration Matrix

Below is a practical mapping tool for AI agents and analysts.

Behavior	Exposure	Exploitability	Business Impact	Severity
SQLi	Public	Easy	DB compromise	Critical

Behavior	Exposure	Exploitability	Business Impact	Severity
XSS	Public	Easy	Session/user compromise	High
IDOR	Auth + URL manipulation	Easy	Data privacy breach	High
File Inclusion	Public	Medium	Config/data exposure	High
Directory Listing	Public	Very Easy	Indirect exposure	Low/Medium
Session Weakness	Authenticated	Medium	Account hijacking	High

This table aligns recon behavior with rational severity.

---

6. Writing Prioritized Remediation With Estimated Effort

Executives and developers NEED realistic effort estimates.

Structure remediation as:

- **Immediate fixes** (quick wins)
- **Short-term fixes** (1–3 days)
- **Long-term hardening** (architecture-level)

Example:

SQL Injection

- Immediate: Block malicious patterns via WAF (hours)
- Short-term: Convert affected queries to prepared statements (1–2 days)
- Long-term: Adopt ORM or parameter-binding frameworks (1–2 weeks)

IDOR

- Immediate: Add defensive logging (hours)
- Short-term: Enforce auth checks (1 day)
- Long-term: Implement centralized authorization middleware (1 week)

XSS

- Immediate: Sanitize reflection points (hours)
- Short-term: Apply HTML encoding library (1 day)
- Long-term: Enforce CSP and templating (2–4 days)

Remediation impact must be balanced with developer fatigue and resource constraints.

---

7. Prompts for AI Agents to Generate Risk Narratives

Below are structured prompts you can use in an AI-powered pentest workflow.

7.1 Prompt — Convert Technical Finding to Business Risk

```
Convert the following technical vulnerability into a business-facing risk narrative.

Include:
- realistic exploitation path
- severity based on exposure + exploitability + impact
- business-level consequences
- prioritized remediation steps with estimated effort
```

7.2 Prompt — Severity Calibration Based on Recon Evidence

```
Based on initial recon signals, assign severity by evaluating:
- public vs authenticated exposure
- ease of exploitation
- potential attacker gain
- organizational impact

Output severity and justification.
```

7.3 Prompt — Remediation Path Planning

```
For each vulnerability class detected in recon:
1. Provide immediate mitigation.
2. Provide short-term remediation with effort estimates.
3. Provide long-term architectural hardening recommendations.
```

Mapping Swagger and Endpoints

1. What Swagger/OpenAPI Represents

Swagger (OpenAPI Specification) provides:

- Full list of endpoints
- HTTP methods (GET, POST, PUT, DELETE, etc.)
- Parameter names, types, constraints
- Authentication/authorization requirements
- Expected request/response bodies

- Error codes
- Object models and schemas
- Relationships between routes
- Hidden or undocumented features (if docs are incomplete or inconsistent)

For a pentester, Swagger is:

> The source-of-truth map for *how the system actually expects to be interacted with.*

This allows us to identify:

- attack surface
- injection points
- object relationships
- privilege boundaries
- weak parameter validation
- mass assignment risks
- IDOR patterns
- undocumented vs documented behavior differences

---

2. Recon Workflow Using Swagger

Here is the professional process used by pentesters and bug bounty hunters.

2.1 Step 1 — Parse All Paths Into an Endpoint List

Swagger documents contain a **paths** object:

```
paths:
  /users/{id}:
    get:
    put:
  /login:
    post:
  /products:
    get:
```

Each path becomes a **canonical endpoint**.

What to extract:

- Full route
- Methods
- Parameter locations (path/query/body/header)

2.2 Step 2 — Identify Parameter Types and Constraints

Swagger tells you things recon alone **cannot**:

- string vs integer
- min/max lengths
- allowed values
- pattern regex
- nullable or required
- object fields

This is gold for:

- fuzzing strategies
- type confusion testing
- mass assignment
- business logic flaws

Swagger example:

```
parameters:
  - name: id
    in: path
    required: true
    schema:
      type: integer
```

Immediately signals potential:

- **IDOR**
- **Forced browsing**
- **Numeric enumeration**

2.3 Step 3 — Extract Request and Response Schemas

Swagger schemas define:

- Required fields
- Optional fields
- Data types
- Hidden internal fields

This allows AI agents to:

- predict vulnerable fields
- test mass assignment (`role` , `isAdmin` , `status`)
- explore object relationships

Example:

```
User:
  id: integer
  email: string
  isAdmin: boolean
```

Red flag: `isAdmin` suggests privilege escalation via mass assignment (CWE-915).

2.4 Step 4 — Extract Authentication Flows

OpenAPI security definitions might reveal:

- API keys
- JWT
- OAuth2 scopes
- BasicAuth
- Bearer tokens

This helps identify:

- privilege escalation routes
- injection surfaces in tokens

- missing auth on sensitive routes
- potential CSRF-like flows in API contexts

Example:

```
security:
  - bearerAuth: []
```

2.5 Step 5 — Compare Swagger vs. Live Recon

One of the most important pentesting tasks:

> Identify endpoints NOT in Swagger.

These are usually:

- deprecated
- admin-only
- hidden internal tools
- test or debug endpoints
- forgotten development routes

This mismatch is a major recon win.

---

3. Classifying Swagger Endpoints in a Pentest Context

After extracting all paths, classify them like any endpoint inventory:

3.1 Public Endpoints

Non-authenticated routes in Swagger.

3.2 Authenticated Endpoints

Routes protected by bearer/API key/OAuth.

3.3 Admin or Privilege-Specific Endpoints

Indicated by:

- scopes
- RBAC fields
- admin URL patterns
- restricted schemas

3.4 API Functional Groups

Organize endpoints by resource type:

- `/users/*`
- `/orders/*`
- `/products/*`
- `/admin/*`

This forms a clean attack surface map.

---

4. Mapping Swagger Endpoints to Pentesting Opportunities

Swagger reveals specific attack surfaces:

4.1 Path Parameters → IDOR / Broken Access Control

Example:

`GET /users/{id}`
Path parameter is user-controlled → test privilege boundaries.

4.2 Query Strings → Injection / Filtering Bypass

Example:

`GET /products?category=shoes`

Test:

- SQLi
- NoSQLi
- filter bypass
- regex injection
- wildcard behaviors

4.3 JSON Bodies → Mass Assignment / Logic Manipulation

Example:

```
{
  "email": "test@test.com",
  "role": "admin"
}
```

Swagger reveals fields the front-end **never shows**.

4.4 Hidden Schema Fields → Horizontal Privilege Abuse

If Swagger lists:

```
"ownerId": 3
```

Test:

- modification
- enumeration
- forced assignment

4.5 File Upload Schemas → Malware/Traversal

Look for:

```
format: binary
```

Potential:

- RCE
- traversal
- parsing vulnerabilities

---

5. Example: Mapping Swagger to Endpoint Recon (Vulnweb Context)

Although vulnweb is not an API-first site, imagine a Swagger describing:

```
/products/{id}
/products
/users/{id}
/login
/orders
```

We would extract:

- numeric ID patterns → enumeration
- object models → hidden fields
- POST bodies → injection vectors
- auth flows → session/token weaknesses

This mapping is identical in real-world APIs.

---

6. AI Prompts for Swagger/Endpoint Mapping

You can use these to automate analysis.

6.1 Prompt — Extract and Normalize Swagger Endpoints

```
You are extracting the attack surface from a Swagger/OpenAPI document.

Task:
1. List all endpoints and HTTP methods.
2. Extract parameters (path, query, body, header).
3. Normalize endpoints into a canonical inventory.
4. Classify by public/auth/admin/API groups.
```

6.2 Prompt — Map Swagger Schemas to Vulnerability Types

```
Given Swagger schemas, identify:
- fields vulnerable to mass assignment
- IDOR indicators
- weak validation
- hidden sensitive fields

Produce a structured security interpretation.
```

6.3 Prompt — Compare Swagger vs Live Recon

```
Using the endpoint inventory and Swagger paths:
1. Identify documented vs undocumented endpoints.
2. Flag discrepancies.
3. Explain security implications of missing or extra paths.
```

6.4 Prompt — Build API Attack Surface Map

```
Using Swagger + manual recon:
1. Map each endpoint to its parameters and schemas.
2. Link to vulnerability types (IDOR, injection, etc.).
3. Build a hierarchical attack surface model.
```

---

7. Summary

In this lesson you learned how to:

- Parse Swagger/OpenAPI to extract the *true backend attack surface*
- Build a canonical endpoint list from documentation
- Classify endpoints into meaningful security zones
- Identify abuse patterns via parameters and schemas
- Detect high-value targets such as mass assignment or IDOR
- Compare Swagger-defined vs actual live endpoints
- Use AI prompts to automate API recon and attack surface modeling

Swagger acts as a **cheat code for recon**—a structured, machine-readable map of the backend that, once interpreted properly, becomes a foundation for pentesting, bug bounty hunting, and AI-driven assessment systems.

OWASP-TOP 10 API Testing

1. Understanding the OWASP API Security Top 10

Here are the categories (2023 version):

1. **API1 – Broken Object Level Authorization (BOLA)**
2. **API2 – Broken Authentication**
3. **API3 – Broken Object Property Level Authorization**
4. **API4 – Unrestricted Resource Consumption**
5. **API5 – Broken Function Level Authorization**
6. **API6 – Unrestricted Access to Sensitive Business Flows**
7. **API7 – Server-Side Request Forgery (SSRF)**
8. **API8 – Security Misconfigurations**
9. **API9 – Improper Inventory Management**
10. **API10 – Unsafe Consumption of APIs**

crAPI contains vulnerabilities across nearly *all* these categories.

---

2. Reconnaissance Workflow for crAPI

Before mapping vulnerabilities, you must extract the API structure.
crAPI exposes routes for:

- Authentication: `/identity/api/auth/*`
- User management: `/identity/api/user/*`
- Vehicles: `/workshop/api/vehicle/*`
- Community posts: `/community/api/*`
- Shop/orders: `/shop/api/*`
- File uploads, image access, verification flows

Initial Recon Goals

- Identify endpoints

- Map authentication flows (JWT)
- Extract object IDs from responses
- Detect user-controlled parameters
- Parse the Swagger doc if provided
- Observe role-based behaviors

---

3. Mapping crAPI to OWASP API Top 10

Below is a detailed and practical mapping.

API1: Broken Object Level Authorization (BOLA)

crAPI is *famous* for BOLA issues.

Common BOLA paths

- **GET /identity/api/user/{userId}**
  Changing `{userId}` returns another user's profile.
- **GET /workshop/api/vehicle/{vehicleId}**
  Vehicle records tied to another user are accessible.

Why it happens

Direct object references with no server-side authorization checks.

Attacker Narrative

A user can enumerate IDs and access other people's PII, vehicle details, and private records.

API2: Broken Authentication

Vulnerable flows

- Weak password reset workflow (`/identity/api/auth/password-reset`)
- Email/OTP logic that can be manipulated
- Tokens not invalidated properly

Targets

- JWT reuse
- Missing refresh token validation
- Token tampering

API3: Broken Object Property Level Authorization

Even when endpoints require login, internal object fields are not protected.

Examples

- Editing profile fields you should not control (e.g., `email_verified`)
- Changing vehicle ownership fields
- Modifying admin-only properties in JSON bodies

API4: Unrestricted Resource Consumption

API allows:

- Heavy enumeration
- Large payload submissions
- Uploading excessively large images
- Repeated password reset attempts

Indicators

No rate limits → brute force, enumeration, denial-of-service patterns.

API5: Broken Function Level Authorization

Certain endpoints are meant for admins but accessible by normal authenticated users.

Examples

- Administrative functions within community or workshop modules
- Performing actions on objects not owned by the user

API6: Unrestricted Access to Sensitive Business Flows

crAPI intentionally exposes:

- Password reset flows exploitable without full identity proof
- Email verification flows vulnerable to tampering
- E-commerce order processing without validation

These flows can be manipulated to bypass security controls entirely.

API7: Server-Side Request Forgery (SSRF)

crAPI contains SSRF-like behaviors in:

- Image retrieval endpoints
- Vehicle metadata automation
- External resource fetching (internal/external URLs)

Test for patterns such as:

```
GET /workshop/api/vehicle/proxy?url=http://127.0.0.1:8080/admin
```

API8: Security Misconfigurations

crAPI exposes multiple misconfigurations:

Common issues

- Verbose error messages
- Stack traces
- Debug info in responses
- CORS misconfigurations
- Open registration with no rate limits
- Weak JWT secret or hardcoded tokens

API9: Improper Inventory Management

This happens when:

- undocumented endpoints exist
- old versions of endpoints are still functional
- internal-only endpoints are exposed publicly

Typical example:

- `/community/api/post/*` may contain hidden debug routes
- `/shop/api/*` exposes versioning inconsistencies

Improper inventory is one of the easiest wins in bug bounty.

API10: Unsafe Consumption of APIs

crAPI interacts with external APIs without validation.

Entry points

- vehicle metadata fetchers
- community content ingestion
- external verification endpoints

Attackers can force the system to consume malicious API responses.

---

4. Example Attack Scenarios Aligned With OWASP API10

Here is how you would test crAPI systematically.

Scenario 1 — Using BOLA to Access Another User's PII

API1 – BOLA

```
GET /identity/api/user/1
GET /identity/api/user/2
```

Observe differences → confirms horizontal privilege flaws.

Scenario 2 — Manipulating Email Verification Flow

API2 – Broken Authentication
API6 – Access to Sensitive Business Flow

Intercept:

```
POST /identity/api/auth/verify-email
```

Alter email/OTP payload → bypass verification.

Scenario 3 — Mass Assignment in Vehicle Update

API3 – Broken Object Property Level Authorization

```
PUT /workshop/api/vehicle/12
{
  "ownerId": 999,
  "vin": "ABC123"
}
```

If server accepts `ownerId`, mass assignment confirmed.

Scenario 4 — Uploading a 500MB Image

API4 – Unrestricted Resource Consumption

Server may allow giant uploads → DoS condition.

Scenario 5 — Accessing Admin-Only Shop Functions

API5 – Broken Function Authorization

Test endpoints:

```
POST /shop/api/admin/order-update
```

Scenario 6 — SSRF Vehicle Lookup

API7 – SSRF

```
GET /workshop/api/vehicle/fetch?url=http://localhost/admin
```

Observe server-side HTTP activity.

Scenario 7 — Discovering Undocumented Endpoints

API9 – Improper Inventory Management

Fuzz:

```
/identity/api/auth/debug
/shop/api/v1/
/community/api/internal/
```

---

5. AI Prompts for OWASP API Top 10 Testing on crAPI

These prompts let agents generate structured recon, classification, and test plans.

5.1 Prompt — Full OWASP API Top 10 Recon

```
Perform OWASP API Top 10 reconnaissance on http://crapi.apisec.ai/.

Tasks:
```

```
- Extract all endpoints through crawling, traffic analysis, and documentation.
- Map each endpoint to OWASP categories (API1–API10).
- Identify high-risk parameters (IDs, object fields, file uploads).
- Produce a structured API attack surface map.
```

5.2 Prompt — BOLA and Access Control Targeting

```
You are analyzing crAPI for BOLA and object-level flaws.

For every endpoint with path parameters, evaluate:
- user ID enumeration potential
- object ownership enforcement
- privilege boundaries

Output a list of likely BOLA vectors.
```

5.3 Prompt — Schema and Mass Assignment Testing

```
Parse the JSON schemas for crAPI.

Identify:
- modifiable sensitive fields
- ownerId, role, status, or privilege attributes
- fields the client should not be allowed to change

Classify each issue using OWASP API3.
```

5.4 Prompt — Business Flow Abuse

```
Analyze crAPI workflows: registration, login, password reset, vehicle management, community posting.
```

```
Identify business flows vulnerable to API6 (sensitive business flow manipulation).

Explain how minimal attacker effort can bypass intended protections.
```

https://www.youtube.com/watch?v=yxB2NvWfriU&t=1s

Web Exploitation Prompts

1. Arbitrary File Access

Definition: Ability to access internal server files by manipulating parameters (e.g., `?file=`).

API Exploitation Angle: APIs exposing file retrieval endpoints (`/download?path=`) are prime targets.

AI Prompt:

```
Analyze all endpoints that accept file paths or filenames.
Attempt benign traversal payloads such as ../, %2e%2e/.
Observe response differences to detect Arbitrary File Access risk.
```

---

2. Binary Planting

Definition: Placing an attacker-controlled binary in a location where the server executes it.

API Angle: APIs allowing file uploads without validation.

AI Prompt:

```
Identify all file-upload endpoints.
Check allowed content types and whether uploaded files become executable.
```

```
Flag endpoints lacking extension or MIME validation.
```

---

3. Blind SQL Injection

Definition: SQLi where responses do not display errors but timing/boolean methods confirm injection.

API Angle: JSON-based queries (`filter` , `sort` , `conditions`) may be injectable.

AI Prompt:

```
Test numeric and boolean query parameters for time-based SQLi using payloads like SLEEP(5).
Observe response delay patterns and infer blind SQL injection.
```

---

4. Blind XPath Injection

Definition: Manipulating XPath statements in XML-backend services.

API Angle: SOAP/XML APIs or XML document queries.

AI Prompt:

```
For XML-handling endpoints, inject XPath boolean payloads.
Compare true/false response times.
Map endpoints where XML data influences backend queries.
```

---

5. Brute Force

Definition: Repeated credential guessing.

API Angle: `/auth/login` without rate limits.

AI Prompt:

```
Identify authentication endpoints.
Check response times, status codes, lockout logic.
Test small-scale rate-limited brute force simulation.
```

---

6. Buffer Overflow Attack

(API rarely vulnerable unless parsing native binaries, but for completeness…)

Prompt:

```
Check endpoints accepting binary file uploads.
Test oversized payloads to determine if backend crashes or leaks errors.
```

---

7. Cache Poisoning

Definition: Manipulating shared cache layers to poison user responses.

API Angle: APIs using CDN or reverse proxy caching.

AI Prompt:

```
Check for user-controlled headers affecting cache keys (X-Forwarded-Host, X-Original-URL).
Test if responses vary with injected header values.
```

---

8. Cash Overflow *(Business logic flaw)*

Spending or accumulating value beyond intended limits.

Prompt:

```
Identify endpoints modifying balances or credits.
Test negative values, decimals, overflow integers.
```

---

9. Clickjacking

API is backend → check UI
But API endpoints can produce HTML emails or templates.

Prompt:

```
Review API-delivered HTML for missing frame-ancestors or X-Frame-Options headers.
```

---

10. Command Injection

Definition: Injecting OS commands.

API Angle: Image processing, PDF rendering, CLI utilities.

AI Prompt:

```
Identify endpoints invoking server-side shell commands.
Inject test payloads like `; sleep 5`.
Observe for execution delay or output anomalies.
```

---

11. Comment Injection

Send data that becomes HTML or JSON comments, influencing downstream parsing.

Prompt:

```
Submit comment-like payloads (<!-- test -->) into user-controlled API fields.
Verify if responses reflect them unescaped.
```

---

12. Content Security Policy (CSP) Weakness

APIs returning dynamic HTML.

Prompt:

```
Inspect API-served HTML for CSP headers.
Identify unsafe directives like unsafe-inline or wildcards.
```

---

13. Content Spoofing

Tricking UI into displaying attacker-controlled content.

Prompt:

```
Enumerate endpoints that return user-generated content.
Inject crafted metadata to modify response representation.
```

---

14. Credential Stuffing

Using leaked credentials across multiple services.

Prompt:

```
Test login endpoint for lack of IP-based throttling or device fingerprinting.
Check if reused passwords trigger any mitigation or alerts.
```

---

15. Cross-Frame Scripting

API delivering UI fragments must restrict iframe usage.

Prompt:

```
Verify presence of anti-clickjacking headers on HTML-serving API endpoints.
```

---

16. Cross-Site History Manipulation (XSHM)

Modifying browser history through API responses.

Prompt:

```
Check API-powered redirects for open redirect or scriptable location changes.
```

---

17. Cross-Site Tracing

TRACE method enabling credential reflection.

API Prompt:

```
Test whether the HTTP TRACE method is enabled.
```

---

18. CSRF (Cross-Site Request Forgery)

Prompt:

```
Check sensitive state-changing endpoints for:
- no CSRF token
- missing SameSite cookie flags
- permissive CORS
```

---

19. Cross-Site Port Attack (XSPA)

Using browser's ability to detect service differences.

Prompt:

```
Test endpoints allowing external URL fetching.
Probe internal ports via difference in response times.
```

---

20. XSS (Cross-Site Scripting)

Prompt:

```
Test JSON/HTML-reflecting API endpoints with payloads:
"><script>alert(1)</script>
Check reflection context and encoding.
```

---

21. Cross-User Defacement

Modifying shared content.

Prompt:

```
Identify social/community endpoints.
Attempt HTML/script injection in shared fields.
```

---

22. Custom Special Character Injection

API fields may break parsers.

Prompt:

```
Inject characters like null bytes, unicode controls, delimiters.
Observe server parsing errors.
```

---

23. Denial of Service

Prompt:

```
Test API behavior against oversized JSON bodies, large arrays, recursive objects.
Check for CPU/Memory spike via complex regex payloads.
```

---

24. Direct Dynamic Code Evaluation (Eval Injection)

Dangerous if API uses eval-like behavior.

Prompt:

```
Identify fields interpreted as expressions.
Test payloads containing quotes, parentheses, executable expressions.
```

---

25. Execution After Redirect (EAR)

Server continues processing after redirect.

Prompt:

```
Check whether side effects happen after receiving 302 location responses.
```

---

26. CORS Exploitation

Prompt:

```
Inspect CORS headers for:
- Access-Control-Allow-Origin: *
- Misused Access-Control-Allow-Credentials
```

---

27. Forced Browsing

Accessing undeclared API routes.

Prompt:

```
Enumerate hidden or undocumented endpoints using:
- common wordlists
- versioned endpoints (/v1, /v2)
- guessing admin/deprecated routes
```

28. Form Action Hijacking

API → usually HTML-based
But API may return templates.

Prompt:

```
Inspect returned HTML for dynamic form actions influenced by user input.
```

---

29. Format String Attack

If API exposes printf-style formatting.

Prompt:

```
Inject %s, %x, %p into string parameters.
Observe formatting anomalies or memory disclosure.
```

---

30. Full Path Disclosure

Server errors reveal absolute paths.

Prompt:

```
Trigger controlled errors.
```

```
Check responses for /var/www/... or C:\inetpub\ paths.
```

---

31. Function Injection

Calling a backend function by manipulating arguments.

Prompt:

```
Look for endpoints accepting function names or callback parameters.
Inject unexpected function identifiers.
```

---

32. Host Header Injection

Prompt:

```
Send crafted Host headers.
Test behaviors like password reset links or URL generation.
```

---

33. HTTP Response Splitting

Based on CRLF injection.

Prompt:

```
Inject %0d%0a sequences into headers/parameters.
Check for multi-header injection.
```

---

34. HTTP Verb Tampering

Prompt:

```
Test same endpoint with OPTIONS, HEAD, PUT, DELETE.
Identify access-control bypass behaviors.
```

---

35. HTML Injection

Prompt:

```
Inject <b>test</b> into text fields.
Check if returned as formatted HTML.
```

---

36. LDAP Injection

Only for LDAP-backed systems.

Prompt:

```
Inject LDAP filter metacharacters (*)(|)(user=*)
Check if authentication outcome changes.
```

---

37. Log Injection

Prompt:

```
Inject newline characters into fields used in logging.
Check logs for manipulation or corruption.
```

---

38. MITM / Man-in-the-Middle

An API issue when TLS is weak.

Prompt:

```
Inspect SSL configuration.
Check if API supports outdated ciphers or insecure TLS versions.
```

---

39. Man-in-the-Browser

Testing client-side, but API may allow session token reuse.

Prompt:

```
Analyze whether hijacked session cookies allow unauthorized calls.
```

---

40. Mobile Code Injection / Non-Final Fields / Object Hijack

Mostly Java-specific.

Prompt:

```
Inspect serialized API objects.
Test unsafe deserialization or code execution flows.
```

---

41. One-Click Attack

Variant of CSRF.

Prompt:

```
Check if GET endpoints perform sensitive state changes.
```

---

42. Parameter Delimiter Abuse

Breaking parsers with special delimiters.

Prompt:

```
Inject semicolons, pipes, null bytes, ampersands.
Observe deviations in parameter parsing.
```

---

43. Page Takeover

API serving dynamic HTML pages.

Prompt:

```
Attempt injection in title/meta/template fields rendered to HTML.
```

---

44. Path Traversal

Prompt:

```
Inject ../ and %2e/ variations.
Check if server reveals unintended files.
```

---

45. Reflected DOM Injection

API returning JavaScript JSON that frontend injects.

Prompt:

```
Identify endpoints returning unescaped JSON injected into the DOM.
Inject payloads and observe client-side script execution.
```

---

46. Regular Expression DoS (ReDoS)

Prompt:

```
Identify regex-validated fields.
Submit payloads like (a+)+ repeated thousands of times.
Observe server CPU spike.
```

---

47. Repudiation Attack

API not logging critical actions.

Prompt:

```
Check whether sensitive endpoints log caller identity and timestamp.
```

---

48. Resource Injection

Inject system resource identifiers (file, port, device).

Prompt:

```
Test endpoints referencing file paths, pipes, sockets.
Inject unexpected values.
```

---

49. Server-Side Includes (SSI) Injection

Prompt:

```
Inject <!--#exec cmd="id"--> into HTML-capable fields.
See if server interprets SSI directives.
```

---

50. Session Fixation

Prompt:

```
Try forcing a predefined session ID.
Check if server accepts it post-login.
```

---

51. Session Hijacking

Prompt:

```
Test reuse of stolen or intercepted tokens.
Check if server binds them to IP/device.
```

52. Session Prediction

Prompt:

```
Analyze session IDs for entropy.
Test incremental or time-based patterns.
```

---

53. Setting Manipulation

Manipulating user settings beyond allowed options.

Prompt:

```
Identify settings endpoints.
Submit unexpected values and observe server acceptance.
```

---

54. Special Element Injection

Inject tokens that break the parser.

Prompt:

```
Inject delimiters, escape characters, encoding edge cases.
```

---

55. SMTP Injection

If API uses email backend.

Prompt:

```
Inject \r\n, extra headers, control characters into email fields.
Observe changes in outbound email format.
```

---

56. SQL Injection

Prompt:

```
Test numeric, string, and JSON body parameters with standard SQL payloads.
Check for error-based, boolean-based, and time-based responses.
```

---

57. SSI Injection

Duplicate above — similar.

---

58. Traffic Flood (DoS)

Prompt:

```
Simulate low-volume repeated requests to detect rate limiting.
```

---

59. Web Parameter Tampering

Prompt:

```
Modify hidden fields or JSON fields not intended for user control.
Compare behavior before/after manipulation.
```

---

60. XPath Injection

Prompt:

```
Inject XPath wildcard conditions.
Observe XML-backed authentication or search responses.
```

---

61. XSRF or SSRF

Prompt:

```
Identify endpoints making server-side HTTP calls.
Inject internal addresses to check if SSRF is possible.
```

https://www.youtube.com/watch?v=MrNObuV3alg

Automated Bug Bounty Exploitation Tricks and Tips with AI Agent

1. Philosophy of AI-Powered Bug Bounty Hunting

Your AI agent is not a scanner.
It is an **analyst**, **pattern recognizer**, and **test generator**.

AI excels at:

- identifying patterns across hundreds of endpoints
- generating payload variations
- crafting exploitation flows
- mapping large attack surfaces
- analyzing JS files, Swagger docs, GraphQL schemas
- producing chainable exploit scenarios
- creating reproducible pentest notebooks

You provide the **intel** → the agent provides **structured reasoning and exploitation paths**.

---

2. High-Impact Automated Recon with AI

Your recon should feed the agent both raw and processed data:

You collect:

- subdomains
- endpoints
- filtered wordlists

- HTTP responses
- JavaScript files
- Swagger/OpenAPI
- parameter lists

The AI agent processes:

- parameter classification
- hidden endpoints
- API method inference
- vulnerability type prediction
- mapping to OWASP/CWE
- chaining opportunities

Automation Trick:
Feed AI the **raw JS of large SPA apps**.
It finds:

- hidden endpoints
- GraphQL queries
- secret parameters
- undocumented admin routes
- internal API naming conventions

---

3. Automated Exploitation Tricks by Category

Here is the **real power section** — how to automate exploitation reasoning.

3.1 SQL Injection Automation

AI Agent Prompt:

```
Given this endpoint and parameter list, generate all SQLi payloads categorized by:
- error-based
- boolean-based
- time-based
- union-based
Generate probing queries that minimally disrupt the app.
```

Automation trick:
Feed the agent **normal response baseline** + **altered response**, and let it detect anomalies automatically.

3.2 IDOR/BOLA Automation

AI Agent Prompt:

```
Given returned JSON objects and ID patterns, enumerate predictable identifiers.
Generate test cases where:
- authenticated user A accesses user B’s objects
- privilege checks are missing
Score each endpoint by IDOR likelihood.
```

Automation trick:
Have the agent automatically identify:

- sequential IDs
- UUID format deviations

- references to “ownerId,” “userId,” “accountId”

This instantly locates high-value IDOR targets.

3.3 XSS Automation

AI Agent Prompt:

```
Analyze reflected responses.
Test for XSS via:
- attribute injection
- script breaking
- HTML context
- markdown parsing
- JSON → DOM insertion
Return an XSS viability matrix per context.
```

Trick:
Give AI **HTML diff output** — it will guess context and payload precision better than humans.

3.4 CORS Exploitation Automation

AI Agent Prompt:

```
Inspect CORS policy from server responses.
Identify:
- wildcard origins
- misused ACA-Credentials
```

```
- reflection of Origin
Generate exploitation workflow for cookie extraction.
```

This instantly reveals cross-domain privilege escalation.

3.5 SSRF Automation

AI Agent Prompt:

```
Identify all endpoints that accept URLs.
Generate SSRF tests for:
- localhost
- cloud metadata
- internal admin dashboards
Construct payload variations with multiple encodings.
```

Trick:
Let agent generate URL obfuscations:

- IPv6
- decimal IP
- nested redirections
- DNS rebinding candidates

3.6 Path Traversal Automation

AI Agent Prompt:

```
Identify all file-based parameters.
Generate traversal payloads for:
- Linux
- Windows
- URL encoding
- double encoding
Craft probing sequences to confirm traversal without exploitation.
```

3.7 CSRF/One-Click Attack Automation

AI Agent Prompt:

```
Identify state-changing endpoints using GET/POST.
Evaluate CSRF exposure:
- no CSRF token
- no SameSite cookie
- CORS misconfiguration
Generate CSRF PoCs.
```

3.8 Cache Poisoning Automation

AI Agent Prompt:

```
Analyze caching headers and shared cache behavior.
Generate cache poisoning attempts using:
- X-Forwarded-Host
```

```
- X-Original-URL
- URL parameter variations
```

3.9 Header Injection (Host, Response Splitting)

AI Agent Prompt:

```
Detect endpoints that reflect headers.
Test CRLF injection and Host header manipulation.
Generate attack variants automatically.
```

3.10 Forced Browsing Automation

AI Agent Prompt:

```
Using known patterns from the app’s JS and naming conventions:
Predict hidden endpoints like:
/admin
/internal
/debug
/v1
/test
Identify unauthorized access attempts.
```

---

4. Automated Attack Chain Generation

The agent should create **multi-step exploitation chains**, not isolated vulnerabilities.

Example Chain:

1. **IDOR** leaks internal info
2. Info enables **password reset bypass**
3. Reset enables **admin takeover**
4. Admin interface exposes **SQLi**

AI Prompt:

```
Using all discovered vulnerability indicators, generate multi-step exploitation chains.
Rank them by realism and impact.
```

---

5. Bug Bounty Automation Prompts (Full Arsenal)

Here is a **complete suite** of prompts for high-speed bug bounty work.

5.1 Universal Recon and Exploitation Prompt

```
You are an automated bug bounty agent.
Given endpoints, parameters, responses, and JS/Swagger data:
1. Classify all endpoints by vulnerability likelihood.
2. Identify potential exploitation vectors.
3. Generate payloads for each vulnerability type.
```

```
4. Prioritize high-impact findings.
5. Build exploitation chains where weaknesses correlate.
```

5.2 Automated Payload Generation

```
Generate payload variations for the following vulnerability type:
[VULN TYPE]
Provide:
- basic payloads
- advanced payloads
- context-specific payloads
- evasion techniques
```

5.3 Endpoint Behavioral Analysis

```
Given the baseline and modified responses:
Identify anomalies including:
- timing differences
- size changes
- HTTP status changes
- JSON key deviations
Infer exploitation potential.
```

5.4 API Abuse Prediction

```
Given object schemas and JSON response fields:
Predict fields that can cause:
- mass assignment
- privilege escalation
- business logic abuse
```

5.5 Multi-Scope Bug Bounty Sweep

```
Using all collected intel:
1. Identify undocumented or hidden endpoints.
2. Detect parameter naming patterns.
3. Predict developer naming conventions.
4. Suggest new endpoints to brute-force.
```

5.6 Automated JS File Exploitation

```
Analyze the JavaScript file and extract:
- hidden routes
- API keys
- sensitive tokens
- debug flags
- commented-out admin code
Generate abuse scenarios.
```

5.7 Risk Interpretation for Submissions

```
Convert each finding into:
- severity
- business impact
- exploitation flow
- remediation guidance
```

---

6. Trick Vault: Tips Top Bug Bounty Hackers Automate

Use AI to Prioritize Weakest Targets

AI identifies endpoints with highest vulnerability density.

Exploit Schema Mismatch

Swagger vs real behavior mismatches → gold mine.

Analyze JS → find feature flags

These reveal:

- beta endpoints
- admin toggles
- hidden functionality

Use AI for Recon Diffs

Feed before/after responses for each fuzzed request.

Parameter Universe Expansion

AI generates hundreds of parameter names used by frameworks:

- `userId`
- `account`
- `role`
- `permissions`
- `debug`

Chain conditions automatically detected by AI

AI can spot:

- missing auth
- parallel object references
- misordered business logic

This leads to **mega bugs**.

---

7. Summary

This lesson delivered:

- Automated bug bounty exploitation tactics
- High-impact AI prompts
- Recon and exploitation workflows
- Vulnerability-specific automation strategies
- Attack chain generation

- Tricks used by elite bounty hunters

You now have the blueprint for building a **self-assisted, AI-driven exploitation workflow**.

Automated Active Directory and Networking Pentesting With an AI Agent

1. Philosophy of Automated AD Pentesting With an AI Agent

Active Directory attacks rely on **patterns**, **relationships**, and **graph structures**.
AI is uniquely suited for this because it can instantly process:

- user-to-group relationships
- ACL misconfigurations
- service principal vulnerabilities
- delegation paths
- privilege escalation flows
- attack graph reasoning

Your AI agent becomes a **BloodHound-style reasoning engine**, but dynamic, conversational, and contextual.

---

2. Automated Network Recon & Asset Discovery

Before AD exploitation begins, networking recon is crucial.

2.1 Automated Passive Recon

AI agent analyzes:

- ARP tables
- DNS zones

- DHCP logs
- SLP/LLMNR/NBT-NS traffic (for poisoning)
- SMB broadcasts
- SNMP enumeration output

2.2 Automated Active Recon

Include:

- Nmap results
- Masscan outputs
- Service banners
- TLS fingerprinting
- Reverse DNS mapping
- Open ports (Kerberos, LDAP, SMB, WinRM, MSSQL)

AI Prompt: Network Recon Interpretation

```
Analyze Nmap/Masscan results.
Identify:
- domain controllers
- Windows hosts
- Linux servers
- open SMB, LDAP, Kerberos, WinRM, MSSQL ports
Predict potential AD attack paths and privileged entry points.
```

---

3. Automated Active Directory Enumeration

AD is about **relationships**, not isolated systems.

3.1 Enumerate Without Credentials

Tools: `ldapsearch`, `crackmapexec`, anonymous binds.

AI should detect:

- domain name
- domain SID
- users with pre-auth disabled
- publicly readable LDAP attributes
- DNS records
- default groups

AI Prompt: Anonymous AD Enumeration

```
Analyze the LDAP enumeration output.
Identify:
- pre-auth disabled users for AS-REP roasting
- LDAP-readable sensitive attributes
- potential user enumeration vulnerabilities
Generate a prioritized list of initial foothold targets.
```

3.2 Enumerate With Low-Priv Credentials

Once any credential is obtained:

- Kerberoastable accounts

- Unconstrained + constrained delegation
- Resource-Based Constrained Delegation (RBCD)
- ACL misconfigurations
- Password policies
- SMB shares
- GPO access

AI Prompt: Domain Enumeration With Credentials

```
Using domain enumeration data:
- Identify SPNs vulnerable to Kerberoasting
- Detect users with weak password policies
- Analyze delegation relationships
- Evaluate ACLs for privilege escalation
Output direct exploitation paths.
```

---

4. Automated Credential Attacks

4.1 AS-REP Roasting

Users with "Do not require Kerberos pre-authentication."

AI interprets dump and generates cracking workflows.

Prompt:

```
Analyze asreproast dump.
Identify users with high-value attributes.
```

```
Generate cracking commands and expected difficulty.
```

4.2 Kerberoasting

SPN accounts.

Prompt:

```
Analyze SPN list.
Determine which accounts are high-value for Kerberoasting.
Generate automated cracking workflow and escalation steps upon success.
```

4.3 Password Spraying

AI designs spraying schedules to evade lockout policies.

Prompt:

```
Given password policy:
Generate a password spraying plan that:
- avoids lockouts
- targets most likely weak users
- prioritizes privileged accounts
```

---

5. Automated Privilege Escalation in AD

5.1 ACL-Based Escalation

- WriteDACL
- GenericAll
- GenericWrite
- AddMember
- ForceChangePassword

AI Agent Prompt:

```
Analyze ACL relationships.
Identify misconfigurations that allow:
- user takeover
- group privilege escalation
- domain privilege escalation
Generate exploitation steps.
```

5.2 Delegation Abuse

- Unconstrained delegation
- Constrained delegation (`msDS-AllowedToDelegateTo`)
- RBCD exploitation

AI Agent Prompt:

```
Examine delegation relationships.
Identify vulnerable hosts and accounts.
Generate abuse workflow for RBCD or constrained delegation.
```

5.3 GPO Abuse

If a low-priv user can modify GPOs → SYSTEM access → Domain Admin.

Prompt:

```
Based on GPO enumeration, identify GPOs writable by non-admin users.
Create exploitation path to execute code on linked hosts.
```

---

6. Automated Local Privilege Escalation on Hosts

AI examines:

- OS versions
- installed software
- missing patches
- misconfigured services
- unquoted service paths
- writable service binaries

AI Prompt:

```
Analyze system enumeration output.
Detect all local privilege escalation vectors.
Prioritize most reliable exploitation paths.
```

---

7. Lateral Movement Automation

Techniques:

- Pass-the-Hash
- Pass-the-Ticket
- Overpass-the-Hash
- DCOM lateral movement
- WinRM / SMBExec
- MSSQL xp_cmdshell abuse

AI Prompt:

```
Given available credentials and host list:
Determine reachable hosts.
Select optimal lateral movement route.
Identify hops leading to high-value systems or DCs.
```

---

8. Automated Domain Dominance

Once privileged credentials are obtained:

- DCSync
- Golden Ticket
- Silver Ticket
- Skeleton Key
- AdminSDHolder abuse
- DNS Admin → DC compromise

AI Prompt:

```
Using Domain Admin or equivalent privileges:
Generate domain dominance actions.
Explain persistence mechanisms and cleanup considerations.
```

---

9. Networking Pentest Exploitation Automation

Beyond AD, AI supports:

9.1 Firewall & ACL Misconfig Identification

Prompt:

```
Analyze firewall rules and routing tables.
Identify segmentation failures and pivot opportunities.
```

9.2 Network Service Exploitation Prediction

AI recognizes vulnerable service combos:

- SMB + old Windows versions → EternalBlue-like
- RDP misconfigurations
- Default SNMP strings
- Weak SSL/TLS

Prompt:

```
Given network service banners, predict matching public exploits.
Generate automated exploitation workflow.
```

9.3 Pivoting Automation

Prompt:

```
Given compromised host network interfaces:
Map reachable internal networks.
Recommend pivot paths and proxychains configuration.
```

---

10. Complete AI Attack Chain for AD (Example)

AI automatically chains:

1. SMB enumeration → exposed user list
2. Password spraying → low-priv account compromise
3. LDAP enumeration → detect SPN
4. Kerberoast → crack service account password
5. ACL misconfig on group → escalate to Domain Admin
6. DCSync → full domain takeover

AI Prompt:

```
Using recon + enumeration data:
Build an exploitation chain to reach Domain Admin.
Justify each step and list required commands.
```

11. Full Prompt Suite — Automated AD + Network Pentest

Here is the **complete operator toolbox**.

11.1 AD & Network Recon

```
Analyze all recon data.
Identify:
- domain controllers
- reachable subnets
- AD enumeration surface
Generate prioritized enumeration plan.
```

11.2 Credential Attacks

```
Interpret LDAP/Kerberos/SPN data.
Identify AS-REP, Kerberoasting, password spraying, and NTLM relay opportunities.
Produce an exploitation workflow.
```

11.3 Privilege Escalation

```
Analyze ACLs, group membership, delegation, and GPO data.
Generate all viable privilege escalation paths.
```

```
Score them by reliability and stealth.
```

11.4 Lateral Movement

```
Given valid credentials:
Identify reachable hosts.
Generate lateral movement strategies and commands.
```

11.5 Domain Dominance

```
Using elevated privileges:
Generate domain takeover strategies such as:
- DCSync
- Golden Ticket
- GPO abuse
- AdminSDHolder backdoor
```

11.6 Pivoting & Internal Network Access

```
Given foothold host details:
Map internal networks.
Determine pivot paths and reachable assets.
```

Automating Phishing, OSINT, and Scanning GitHub Repositories for Secrets and Backdoors with AI

1. Automating Phishing with AI

Phishing is a highly effective attack vector for gaining initial access, particularly in **bug bounty** or **red team** operations.
AI can help automate the process of **crafting phishing emails**, **spoofing legitimate domains**, and **tracking user responses**. Here's how you can scale phishing attacks using AI.

1.1 Phishing Email Campaign Automation

An AI agent can generate custom phishing emails by **scraping public sources** like **social media profiles** or **company websites**, then **personalizing** messages based on the data it finds.

AI Prompt for Phishing Email Generation:

```
You are an AI phishing specialist.
Task:
1. Given a target email address, gather publicly available data from:
   - LinkedIn
   - Facebook
   - Twitter
   - GitHub
2. Create a realistic phishing email mimicking a service relevant to the target's interests.
3. Include:
   - customized greetings
   - a credible call to action (e.g., password reset, account verification)
   - links to fake login pages
4. Track responses and click-through rates.
```

1.2 Phishing Site Creation Automation

The AI agent can automatically generate **fake landing pages** that mimic popular services (like Google, Microsoft, or Facebook). It can set up **hidden backends** to **capture user credentials** and send them to a remote server.

AI Prompt for Phishing Page Generation:

```
You are generating phishing landing pages.
Task:
1. Based on the target’s interests (from OSINT), create a realistic clone of a website's login form.
2. Ensure the backend captures credentials.
3. Use domain-spoofing techniques to mask the phishing page.
4. Implement tracking and reporting for user interactions.
5. Use HTTPS to appear legitimate.
```

---

2. Automating OSINT Collection

Open Source Intelligence (OSINT) is the process of gathering publicly available information from various sources. The AI can automate this process to collect useful details about the target, such as email addresses, social profiles, infrastructure information, and more.

2.1 Automating OSINT Collection Using Public Databases

AI tools can scan public repositories, social media profiles, and databases like:

- **HaveIBeenPwned** (for breached accounts)
- **Shodan** (for exposed devices)
- **Social Media** (Twitter, Facebook, LinkedIn, etc.)
- **WHOIS databases**
- **Public Code Repositories** (GitHub, GitLab)

AI Prompt for OSINT Collection:

```
You are an OSINT automation agent.
Task:
```

```
1. Search for the target's email address across:
   - HaveIBeenPwned
   - LinkedIn, Twitter, Facebook
   - GitHub (public repos)
2. Extract:
   - associated usernames
   - past breaches
   - contact details
   - exposed infrastructure or tech stack
3. Generate a structured report containing all gathered data.
```

2.2 Automating Social Media Profiling

AI can scrape social media platforms to gather information about a target's:

- Friends and associates
- Work history
- Interests and hobbies
- Locations and schedules

AI Prompt for Social Media OSINT:

```
You are an AI social media profiler.
Task:
1. Scrape the following social platforms for the target:
   - LinkedIn
   - Facebook
   - Instagram
2. Extract:
   - Work history
   - Connections and followers
```

```
   - Places the target has visited (check-in data)
3. Generate a user profile with useful information for crafting personalized phishing emails.
```

---

3. Automating GitHub Repositories and Codebase Scanning for Secrets and Backdoors

Many organizations host sensitive code, API keys, database credentials, or backdoors in their public GitHub repositories.
AI agents can automate the scanning of **GitHub repositories** and **codebases** for common secrets like:

- API keys
- AWS/GCP credentials
- Database connection strings
- Hardcoded passwords
- Backdoors in commit history

3.1 GitHub Repository Scanning for Secrets

AI agents can scan **public repositories** for sensitive data. By using **regex** patterns or predefined keyword sets, it can flag potentially dangerous items. It can also cross-check this data against a known database of leaked credentials.

AI Prompt for GitHub Secret Scanning:

```
You are a GitHub codebase scanner.
Task:
1. Scan the following GitHub repositories for the following types of secrets:
   - AWS keys
   - Google Cloud API tokens
   - Database credentials (MongoDB, MySQL, Redis)
   - Hardcoded passwords or secrets in plaintext
```

```
2. Flag files containing sensitive data and classify by severity.
3. Output a report with all instances of exposed secrets and recommended remediation.
```

3.2 Backdoor Detection in Code Repositories

AI can also help detect **backdoors** or **debugging code** (e.g., `eval()`, `exec()`, `system()`, `os.popen()`), which are often inserted into repositories by developers but may lead to vulnerabilities.

AI Prompt for Backdoor Detection:

```
You are scanning a codebase for backdoors.
Task:
1. Search for the following patterns:
   - exec(), eval(), system(), os.popen()
   - Hardcoded admin credentials
   - GitHub Actions with insecure environments
2. Flag any instances where dangerous code is found and generate exploitation scenarios.
3. Output remediation steps to remove the backdoor or refactor unsafe code.
```

---

4. Generating Exploitation Chains from OSINT and GitHub Scans

Once you've gathered OSINT data and identified secrets or vulnerabilities in code repositories, the AI can automatically generate **exploitation chains** that outline how an attacker would use this information to gain access.

4.1 Building Exploitation Chains

The AI generates a sequence of events that lead from **recon** → **exploitation** → **lateral movement**:

1. **Recon:** Target found on LinkedIn, email breached in previous attacks
2. **Phishing:** Craft email with information gathered from social media
3. **Exploit:** Use leaked API keys from GitHub to access backend services
4. **Escalate:** Use exposed credentials to escalate privileges and move laterally

AI Prompt for Exploitation Chain Generation:

```
Given the OSINT and GitHub secrets data:
1. Build an attack chain that includes:
   - Initial access (phishing or credential stuffing)
   - Privilege escalation (via exposed secrets)
   - Lateral movement (via API key exploitation or database access)
2. Provide a step-by-step exploitation plan with command examples.
```

---

5. Ethical and Legal Considerations

Automating exploitation is powerful, but **always stay within legal boundaries**.
Ensure that your AI-driven automation tools only target systems where you have **explicit permission** to test, whether that's:

- Authorized bug bounty programs
- Your own systems
- Penetration testing engagements with prior consent

Remember: Unauthorized scanning or exploitation can lead to **severe legal consequences**.

Model Context Protocol (MCP) Fundamentals for Cybersecurity

Lesson Objective

This lesson introduces the **Model Context Protocol (MCP)** from a cybersecurity perspective.
By the end of this lesson, the student will understand **what MCP is**, **why it matters for security operations**, and **how to reason about MCP architectures safely** before integrating it into offensive or defensive workflows.

This lesson is **conceptual and architectural**. No tooling is installed yet.
The focus is to build the correct mental model before touching real systems.

---

1. What Problem MCP Solves in Cybersecurity

Modern cybersecurity tooling suffers from a recurring problem:

- Tools do not talk to each other naturally
- Automation logic is glued together with brittle scripts
- Context is lost between tools (scope, findings, assumptions)
- AI assistants hallucinate because they lack real access to tools and data

MCP exists to solve **context fragmentation**.

Instead of an AI guessing, MCP allows the AI to:

- Call tools explicitly
- Read structured data
- Respect clear boundaries between reasoning and execution

In cybersecurity, this is critical because:

- Tools can **scan**, **modify**, or **exfiltrate data**
- Mistakes can cause **legal**, **ethical**, or **operational damage**

MCP provides a **controlled interface** between AI reasoning and real-world actions.

2. High-Level Definition of MCP

Model Context Protocol (MCP) is a standardized protocol that allows AI applications to connect to external tools, data sources, and environments through a well-defined interface.

You can think of MCP as:

> A secure contract that defines *what* an AI can access, *how* it can access it, and *what type of outputs* it will receive.

This contract-based model is essential in security-sensitive environments.

---

3. Core MCP Architecture

An MCP system is composed of **four fundamental components**:

3.1 MCP Client

The **client** is the AI application itself.

Examples in this course:

- Claude Code
- Gemini-based custom clients
- IDE-integrated AI agents

The client:

- Performs reasoning
- Decides when a tool is needed
- Sends structured requests to MCP servers

The client **does not directly execute system commands**.

3.2 MCP Server

The **server** exposes capabilities to the client.

It is usually:

- A local process
- Or a remote service

The server defines:

- What tools exist
- What data can be accessed
- What inputs are valid
- What outputs look like

From a security standpoint, the MCP server is **the enforcement point**.

3.3 Tools

Tools are actions the AI can request.

In cybersecurity, examples include:

- Running a scan with predefined flags
- Querying findings from a database
- Parsing a binary or configuration file

- Summarizing logs
- Validating scope rules

Important rule:

> Tools should be **explicit, narrow, and auditable**

A good MCP tool:

- Has a clear purpose
- Has strict input validation
- Produces structured output

3.4 Resources

Resources are data that the AI can read.

Examples:

- Files
- Reports
- Decompiled functions
- Scan results
- Configuration artifacts

Resources are typically:

- Read-only
- Versioned
- Scoped to a project

Resources prevent hallucination by grounding the AI in **real data**.

---

4. MCP Transports (How Communication Happens)

MCP supports multiple communication transports.
Understanding these is essential for secure deployments.

4.1 STDIO Transport (Local)

- The client launches the server as a local process
- Communication happens via standard input/output

Security characteristics:

- Local only
- Ideal for reverse engineering tools
- Ideal for developer machines
- Lower attack surface

Typical use cases:

- IDA Pro integration
- Local static analysis
- Developer automation

4.2 HTTP Transport (Remote)

- The MCP server exposes an HTTP endpoint

- The client connects over the network

Security characteristics:

- Higher risk
- Requires authentication and access control
- Must assume hostile networks

Typical use cases:

- Burp Suite integration
- Centralized scanning services
- SOC or platform environments

Rule of thumb:

> Use STDIO whenever possible.
> Use HTTP only when absolutely necessary.

---

5. Why MCP Is a Game-Changer for Cybersecurity

MCP enables **agentic security workflows** without chaos.

Instead of:

- “Run this script, then parse output, then copy-paste into a report”

You get:

- AI reasoning
- Tool invocation

- Structured findings
- Traceable decisions

Key benefits:

- Reduced human error
- Consistent outputs
- Better auditability
- Clear trust boundaries

This is especially valuable in:

- Red Team operations
- Malware analysis
- Web application testing
- Threat hunting
- Incident response

---

6. Trust Boundaries and Threat Model

6.1 MCP Is Code Execution

Never forget:

> An MCP server is effectively a privileged execution interface.

If compromised, it can:

- Execute commands
- Leak data

- Modify systems

Therefore:

- Treat MCP servers like production services
- Apply least privilege
- Isolate them whenever possible

6.2 Prompt Injection and Tool Abuse

AI reasoning can be manipulated through:

- Malicious input data
- Untrusted web content
- Poisoned resources

Defensive strategies:

- Never expose raw command execution
- Enforce allowlists
- Validate every input schema
- Log every tool call

6.3 Separation of Environments

Always separate:

- Lab environments

- Client environments
- Production environments

Never reuse:

- API keys
- MCP configs
- Tool permissions

Between environments.

---

7. Cybersecurity Mental Model for MCP

Before integrating MCP into any security workflow, always ask:

1. What authority does this tool have?
2. What data can it access?
3. What happens if the AI is wrong?
4. Can this action be audited?
5. Can this be abused?

If you cannot answer these questions clearly, **do not deploy MCP yet**.

---

8. Lesson Summary

In this lesson, you learned:

- What MCP is and why it matters in cybersecurity

- The difference between clients, servers, tools, and resources
- How MCP transports affect security posture
- Why MCP must be treated as privileged infrastructure
- How to reason about trust boundaries before automation

Hands-On MCP with Claude Code for Cybersecurity Workflows

Lesson Objective

This lesson moves from theory to practice.

By the end of this lesson, the student will:

- Understand how **Claude Code** works as an MCP client
- Know how MCP servers are registered, scoped, and managed
- Learn how to integrate **security tools** safely using MCP
- Build a controlled mental model for **authorized web security workflows**

This lesson is still **safe-first**.
We focus on **integration, visibility, and control**, not exploitation.

---

1. Claude Code as an MCP Client

Claude Code is a developer-oriented AI environment designed to:

- Work directly with codebases
- Invoke tools explicitly
- Respect structured interfaces instead of free-form guessing

In MCP terms:

- Claude Code = **MCP Client**
- External tools = **MCP Servers**

Claude Code **never executes tools directly**.
It sends **structured requests** and receives **structured responses**.

This separation is critical in cybersecurity.

---

2. How Claude Code Thinks About MCP

Claude Code treats MCP servers as:

- Named capabilities
- With explicit transport types
- With defined scopes

Each MCP server:

- Is registered manually
- Has a unique name
- Can be enabled or removed at any time

This prevents:

- Accidental execution
- Shadow automation
- Silent privilege escalation

---

3. MCP Server Transports in Claude Code

Claude Code supports two primary MCP server types:

3.1 STDIO Servers (Local)

Characteristics:

- Server is launched as a local process
- Communication via stdin/stdout
- No network exposure

Security posture:

- Lowest risk
- Best for reverse engineering
- Best for local automation
- Ideal for labs and research

Typical security use cases:

- Binary analysis tools
- Static analyzers
- Custom parsers
- Local data processing

3.2 HTTP Servers (Remote)

Characteristics:

- Server exposes an HTTP endpoint
- Client connects over network

Security posture:

- Higher risk
- Must be authenticated
- Must be isolated

Typical security use cases:

- Proxy-based testing tools
- Centralized scanning services
- SOC-style platforms

Rule:

> If a tool can run locally, prefer STDIO.

---

4. Managing MCP Servers in Claude Code

Claude Code provides explicit commands to manage MCP servers.

Conceptually, you can:

- Add a server
- List all servers
- Inspect a server
- Remove a server

This explicit lifecycle is intentional.

From a security standpoint:

- Nothing runs implicitly
- Everything is visible
- Everything is reversible

This aligns well with professional security workflows.

---

5. Example Security Workflow: Web Testing (Conceptual)

We will use **web application testing** as the reference workflow.

Important constraint:

> Only test applications you own or have explicit permission to test.

5.1 Traditional Workflow (Without MCP)

1. Configure proxy
2. Capture traffic
3. Run scans
4. Export results
5. Manually summarize findings
6. Write report

Problems:

- Context loss

- Manual errors
- Inconsistent severity mapping
- Time-consuming reporting

5.2 MCP-Enabled Workflow (With Claude Code)

1. Proxy tool runs as MCP server
2. Findings are exposed as structured data
3. Claude Code queries findings
4. Claude Code reasons over scope and impact
5. Claude Code generates a structured summary

Key difference:

- Claude does **not scan blindly**
- Claude only **consumes authorized outputs**

This is the correct use of AI in security.

---

6. Tool Design Principles for Security MCP Servers

When integrating any security tool via MCP, enforce these rules.

6.1 Narrow Tool Scope

Bad tool:

- “Run any command”

Good tool:

- “Fetch findings for target X”
- “Summarize issues of severity HIGH”

Always prefer **high-level abstractions**.

6.2 Structured Inputs Only

Inputs should:

- Be typed
- Be validated
- Reject unexpected fields

Never accept:

- Raw shell strings
- Unvalidated paths
- Arbitrary flags

6.3 Structured Outputs Only

Outputs should:

- Be JSON or well-defined objects
- Include metadata (time, target, tool)
- Be reproducible

This allows:

- Auditing
- Comparison
- Reporting automation

---

7. Security Guardrails in Claude Code

7.1 Least Privilege

MCP servers should:

- Run as non-root users
- Have minimal filesystem access
- Have limited network access

Claude Code does not enforce this automatically.
You must.

7.2 Logging and Auditing

Every MCP tool call should be logged:

- Tool name
- Inputs
- Outputs
- Timestamp

If you cannot explain what the AI did:

> You should not be using AI in that workflow.

7.3 Environment Separation

Never mix:

- Personal lab tools
- Client tools
- Production tools

Create:

- Separate MCP configurations
- Separate API keys
- Separate execution environments

This prevents catastrophic mistakes.

---

8. Common Mistakes When Using MCP with Claude Code

Avoid these patterns:

- Giving the AI raw shell access
- Letting the AI decide scan scope
- Feeding untrusted web content into tools
- Using the same MCP setup across environments

- Treating MCP as “just automation”

MCP is **infrastructure**, not a script.

---

9. Mental Checklist Before Adding Any MCP Server

Before integrating a new MCP server, ask:

1. What can this server execute?
2. What data can it access?
3. Can it modify state?
4. Is output deterministic?
5. Is this action reversible?
6. Is this authorized?

If any answer is unclear:

> Stop and redesign.

---

10. Lesson Summary

In this lesson, you learned:

- How Claude Code functions as an MCP client
- The difference between STDIO and HTTP servers in practice
- How MCP enables safer security automation
- Why explicit tool design matters in cybersecurity

- How to apply least privilege and auditing to AI workflows

MCP with Gemini for Cybersecurity Automation and AI Agents

Lesson Objective

This lesson focuses on **using MCP with Google Gemini** in cybersecurity contexts.

By the end of this lesson, the student will:

- Understand how Gemini fits into the MCP ecosystem
- Learn the two main Gemini + MCP integration models
- Design safe Gemini-driven security automation
- Compare **Gemini MCP workflows** with **Claude Code MCP workflows**
- Build a correct mental model for Gemini-based security agents

This lesson is **architecture + engineering focused**.
The emphasis is on **designing reliable AI-assisted security systems**, not shortcuts.

---

1. Where Gemini Fits in MCP Architectures

Gemini is a **general-purpose reasoning model**, similar in capability to Claude, but commonly used in:

- Cloud-native environments
- Backend automation
- API-driven systems
- Data processing pipelines

In MCP terms:

- Gemini is **not special**
- It acts as a **client-side reasoning engine**
- MCP defines how Gemini interacts with tools and data

Important distinction:

> MCP controls **capabilities**, Gemini controls **reasoning**.

---

2. Two Valid Gemini + MCP Integration Models

There are **two primary architectures** for using Gemini with MCP in cybersecurity.

---

3. Model A — Gemini as an MCP Client (Direct Tool Control)

3.1 Architecture Overview

In this model:

- Gemini is the AI brain
- MCP servers expose security tools
- Gemini calls MCP tools via structured interfaces

Flow:

1. Gemini receives a task
2. Gemini reasons about required tools
3. Gemini invokes MCP tools
4. Tools return structured output

5. Gemini synthesizes results

This is similar to how Claude Code works, but **you control the client logic**.

3.2 When This Model Makes Sense

Use this model when:

- You are building a **custom security platform**
- You need full control over AI behavior
- You want to embed Gemini into pipelines
- You want deterministic automation

Examples:

- Automated triage of scan results
- Malware analysis assistants
- Threat hunting copilots
- Configuration review engines

3.3 Security Implications

Advantages:

- Full control
- Fine-grained permissions
- Easier auditing

Risks:

- You must implement guardrails yourself
- Poor tool design can lead to abuse
- Requires engineering discipline

Rule:

> This model is powerful but unforgiving.

---

4. Model B — Gemini Exposed via an MCP Server (Reasoning as a Service)

4.1 Architecture Overview

In this model:

- Gemini runs behind an MCP server
- Other MCP clients can query Gemini
- Gemini acts as a **reasoning service**

Flow:

1. Client invokes MCP tool
2. MCP server calls Gemini API
3. Gemini produces reasoning or analysis
4. Structured output is returned

Gemini becomes a **capability**, not the controller.

4.2 When This Model Makes Sense

Use this model when:

- You already have an MCP client
- You want Gemini as a shared service
- You want to standardize AI reasoning
- You want to isolate API keys

Examples:

- “Summarize findings” service
- “Explain binary behavior” service
- “Normalize vulnerability descriptions” service
- “Generate remediation guidance” service

4.3 Security Implications

Advantages:

- API keys isolated
- Centralized policy enforcement
- Easier rotation and monitoring

Risks:

- Over-centralization
- Abuse via excessive requests
- Dependency on a single reasoning service

Rule:

> Treat Gemini MCP servers like production APIs.

---

5. Designing Gemini MCP Tools for Cybersecurity

Regardless of the model, **tool design determines safety**.

5.1 Good Gemini MCP Tool Characteristics

A good tool:

- Performs **one task**
- Has **strict input schema**
- Produces **structured output**
- Is deterministic when possible

Examples:

- `summarize_findings(findings_json)`
- `classify_risk(vulnerability)`
- `extract_iocs(report_text)`

5.2 Bad Tool Characteristics

Avoid tools that:

- Execute arbitrary commands
- Accept free-form shell input
- Make network requests without validation
- Modify systems by default

Example of a dangerous tool:

- `run_command(command_string)`

This is unacceptable in professional environments.

---

6. Gemini MCP in Security Automation Pipelines

6.1 Read-Only First Principle

Start with:

- Analysis
- Classification
- Explanation
- Correlation

Avoid starting with:

- Scanning
- Exploitation
- State changes

Gemini excels at **thinking**, not **acting**.

6.2 Example Safe Pipeline

1. Scanner produces JSON findings
2. MCP resource exposes findings
3. Gemini analyzes severity patterns
4. Gemini maps issues to risk categories
5. Output is reviewed by a human

This keeps humans in control.

---

7. Gemini MCP and Prompt Injection Risk

Gemini reasoning can be influenced by:

- Malicious input data
- Crafted payloads in logs
- Embedded instructions in reports

Defensive strategies:

- Never trust external content blindly
- Strip markup and executable content
- Validate input formats
- Enforce length and type constraints

Important rule:

> Treat all input to Gemini as untrusted data.

---

8. Gemini MCP vs Claude Code MCP

Aspect	Claude Code	Gemini MCP
Primary use	Developer workflows	Automation / platforms
Integration style	Interactive	Programmatic
Guardrails	Built-in UX	Must be implemented
Best for	Analysts, engineers	Pipelines, services
Risk level	Lower by default	Depends on design

Neither is “better”.
They serve **different operational needs**.

---

9. Operational Hardening Checklist

Before deploying Gemini MCP in real environments:

- Separate API keys per environment
- Limit request rates
- Enforce schema validation
- Log all tool calls
- Monitor usage patterns
- Isolate MCP servers
- Assume breach scenarios

If you cannot monitor it:

> You cannot trust it.

---

10. Lesson Summary

In this lesson, you learned:

- How Gemini fits into MCP architectures
- The two main Gemini + MCP integration models
- How to design safe MCP tools for Gemini
- Why Gemini should focus on reasoning, not execution
- How Gemini MCP compares to Claude Code MCP
- How to reduce risk in AI-driven security automation

Configuring Pentesting Tools with MCP (Burp, IDA, Ghidra, JADX, Kali)

Configuring Pentesting Tools with MCP (Burp, IDA, Ghidra, JADX, Kali)

Lesson Objective

This lesson is **fully practical**.

By the end of this lesson, the student will:

- Understand how to expose **pentesting tools** via MCP safely
- Configure MCP servers for common security tools
- Design tool boundaries per environment (lab, client, research)
- Avoid common misconfigurations that lead to over-privileged AI agents

This lesson assumes:

- MCP fundamentals are understood (Lessons 1–3)
- You are working in **authorized environments only**

---

1. Core Principle Before Configuration

Before configuring any tool with MCP, internalize this rule:

> MCP does not make tools safer.
> **You make tools safer by how you expose them through MCP.**

Therefore:

- Never expose raw shells
- Never expose full tool UIs
- Never expose unrestricted file systems

Every configuration below follows this rule.

---

2. Burp Suite MCP Configuration (Web Pentesting)

2.1 Purpose of Burp MCP

Burp MCP should be used for:

- Reading findings
- Querying scan results
- Summarizing issues
- Correlating endpoints

It should **not**:

- Start scans
- Modify scope
- Change Burp configuration dynamically

2.2 Burp MCP Conceptual Configuration

Transport

- HTTP (local only)

Bind

- `127.0.0.1`
- Never `0.0.0.0`

Port

- Fixed, known port (example: `9876`)

Allowed Capabilities

- Read-only access to:
  - Issues
  - Requests
  - Responses
  - Scan metadata

Disallowed Capabilities

- Config editing
- Scan execution
- Intruder automation
- Extensions management

2.3 Claude Code MCP Registration (Conceptual)

```
claude mcp add burp \
  --transport http \
  --scope local \
  http://127.0.0.1:9876
```

Security Notes

- Keep Burp and Claude on the same machine
- Do not tunnel this port
- Do not expose Burp MCP to cloud environments

---

3. IDA Pro MCP Configuration (Reverse Engineering)

3.1 Purpose of IDA MCP

IDA MCP is ideal for:

- Function summaries
- Cross-reference analysis
- String extraction

- Control-flow explanation

IDA MCP should **never**:

- Patch binaries
- Execute samples
- Modify databases automatically

3.2 IDA MCP Execution Model

Transport

- STDIO (local process)

Execution Context

- Same user as IDA
- Same working directory

Access Scope

- IDA database only
- Read-only operations

3.3 Example STDIO MCP Server Pattern

```
{
  "mcpServers": {
```

```
    "ida": {
      "type": "stdio",
      "command": "ida-pro-mcp",
      "args": [],
      "env": {}
    }
  }
}
```

Security Notes

- Never run IDA MCP as root
- Use isolated VMs for malware analysis
- Snapshot before loading samples

---

4. Ghidra MCP Configuration (Static Analysis)

4.1 Purpose of Ghidra MCP

Ghidra MCP is useful for:

- Decompiled function review
- Symbol explanation
- Data-flow summaries
- Architecture detection

4.2 Ghidra MCP Execution Model

Transport

- STDIO

Invocation

- Wrapper script launching Ghidra headless
- MCP server reads project data

Allowed Operations

- Read-only decompilation
- Symbol listing
- Function metadata

Disallowed Operations

- Auto-patching
- Script execution
- Project modification

4.3 Safe Ghidra MCP Wrapper (Conceptual)

```bash
#!/bin/bash
export GHIDRA_PROJECT_PATH=/analysis/projects
export GHIDRA_HEADLESS=true
ghidra-mcp-server
```

Security Notes

- Use headless mode
- Never allow dynamic script execution
- Restrict project directory permissions

---

5. JADX MCP Configuration (Android Analysis)

5.1 Purpose of JADX MCP

JADX MCP is ideal for:

- Decompiled Java/Kotlin review
- Manifest analysis
- Permission mapping
- API usage explanation

5.2 JADX MCP Execution Model

Transport

- STDIO

Inputs

- APK path (validated)
- Analysis flags (fixed set)

Outputs

- Decompiled classes
- Structured summaries

5.3 Safe Tool Interface Design

Good tool:

- `analyze_apk(apk_path)`

Bad tool:

- `run_jadx(args)`

Example conceptual config:

```
{
  "mcpServers": {
    "jadx": {
      "type": "stdio",
      "command": "jadx-mcp",
      "args": [],
      "env": {}
    }
  }
}
```

Security Notes

- Validate APK paths
- Disallow network access

- Use temporary analysis directories

---

6. Kali MCP Server Configuration (High-Risk)

6.1 Important Warning

> Kali MCP servers are **inherently dangerous**.

They can:

- Execute terminal commands
- Access system tools
- Modify network state

Use **only in labs**.

6.2 Kali MCP Server Use Cases

Allowed:

- Running predefined recon tools
- Parsing outputs
- Listing tool versions

Not allowed:

- Arbitrary shell execution
- Privilege escalation

- Live client environments

6.3 Safe Kali MCP Capability Design

Instead of:

- `execute(command)`

Expose:

- `run_nmap(target, profile)`
- `run_httpx(target)`
- `list_open_ports(report_id)`

6.4 Isolation Requirements

Mandatory:

- Dedicated VM
- No shared credentials
- No VPN to corporate networks
- Snapshot before usage

---

7. Environment Separation Strategy

Create **separate MCP profiles**:

Environment	Tools Allowed
Lab	All (isolated)
Research	IDA, Ghidra, JADX
Client	Burp (read-only), reports
Production	None

Never reuse:

- MCP configs
- API keys
- Tool servers

---

8. Common Misconfigurations (Avoid These)

- Exposing shell access
- Allowing config edits
- Running MCP servers as root
- Mixing tools across environments
- Trusting AI-generated scope

Each mistake can lead to:

- Data leaks
- Legal exposure
- Client impact

---

9. Operational Checklist

Before enabling any MCP tool:

- Is this read-only?
- Is execution limited?
- Is access logged?
- Is scope enforced?
- Is rollback possible?

If any answer is “no”:

> Do not enable the tool.

---

10. Lesson Summary

In this lesson, you learned how to:

- Safely configure pentesting tools with MCP
- Apply least privilege to Burp, IDA, Ghidra, JADX, and Kali
- Choose correct transports per tool
- Avoid high-risk MCP patterns
- Treat MCP as privileged infrastructure

References

Osama, M. (2023). *HexStrike AI*. Retrieved from https://lnkd.in/dkvmMCDM
NahamSec. (2023). *No Code Bug Bounty Automation Using n8n*. Retrieved from https://lnkd.in/dviUMsRy
Hammond, J. (2023). *I Had Claude MCP Hack Me*. Retrieved from https://lnkd.in/dV5EbcDi
Tech Raj. (2023). *Claude Code vs HackTheBox*. Retrieved from https://lnkd.in/drjJZyA5

The Exploit Lab. (2025). *Kali Linux 2025.3 — gemini-cli*. Retrieved from https://lnkd.in/dPbKptHR

Bombal, D. (2023). *AI + Metasploit*. Retrieved from https://lnkd.in/dMrZsJVX

Network Chuck. (2024). *You Need to Learn MCP RIGHT NOW*. Retrieved from https://lnkd.in/dKfjjb%5Fv

CyberSecurityUP. (2023). *Offensive AI Agent Prompts (GitHub)*. Retrieved from https://github.com/CyberSecurityUP/Offensive-AI-Agent-Prompts

DVWA AI Agent Development to Pentest. (2024). Retrieved from https://lnkd.in/dqY%5FRq7z

Xalgord. (2024). *Gemini-Cli — Cyber Security*. Retrieved from https://lnkd.in/dJRgGHwc